

C++ Standard Library Issues to be moved in Kona

Doc. no. P0165R0

Date: Revised 2015-10-23 at 22:10:34 UTC

Project: Programming Language C++

Reply to: Marshall Clow <lwgchair@gmail.com>

Ready Issues

[1169](#). `num_get` not fully compatible with `strt*`

Section: 22.4.2.1.2 [facet.num.get.virtuals] **Status:** [Ready](#) **Submitter:** Cosmin Truta **Opened:** 2009-07-04 **Last modified:** 2015-05-22

View other [active issues](#) in [facet.num.get.virtuals].

View all other [issues](#) in [facet.num.get.virtuals].

View all issues with [Ready](#) status.

Discussion:

As specified in the latest draft, [N2914](#), `num_get` is still not fully compatible with the following C functions: `strtoul`, `strtoull`, `strtof` and `strtod`.

In C, when conversion of a string to an unsigned integer type falls outside the representable range, `strtoul` and `strtoull` return `ULONG_MAX` and `ULLONG_MAX`, respectively, regardless whether the input field represents a positive or a negative value. On the other hand, the result of `num_get` conversion of negative values to unsigned integer types is zero. This raises a compatibility issue.

Moreover, in C, when conversion of a string to a floating-point type falls outside the representable range, `strtof`, `strtod` and `strtold` return `±HUGE_VALF`, `±HUGE_VAL` and `±HUGE_VALL`, respectively. On the other hand, the result of `num_get` conversion of such out-of-range floating-point values results in the most positive/negative representable value. Although many C library implementations do implement `HUGE_VAL` (etc.) as the highest representable (which is, usually, the infinity), this isn't required by the C standard. The C library specification makes no statement regarding the value of `HUGE_VAL` and friends, which potentially raises the same compatibility issue as in the above case of unsigned integers. In addition, neither C nor C++ define symbolic constants for the maximum representable floating-point values (they only do so only for the maximum representable *finite* floating-point values), which raises a usability issue (it would be hard for the programmer to check the result of `num_get` against overflow).

As such, we propose to adjust the specification of `num_get` to closely follow the behavior of all of its underlying C functions.

[2010 Rapperswil:]

Some concern that this is changing the specification for an existing C++03 function, but it was pointed out that this was underspecified as resolved by issue 23. This is clean-up for that issue in turn. Some concern that we are trying to solve the same problem in both clause 22 and 27.

Bill: There's a change here as to whether val is stored to in an error case.

Pablo: Don't think this changes whether val is stored to or not, but changes the value that is stored.

Bill: Remembers having skirmishes with customers and testers as to whether val is stored to, and the resolution was not to store in error cases.

Howard: Believes since C++03 we made a change to always store in overflow.

Everyone took some time to review the issue.

Pablo: C++98 definitely did not store any value during an error condition.

Dietmar: Depends on the question of what is considered an error, and whether overflow is an error or not, which was the crux of LWG 23.

Pablo: Yes, but given the "zero, if the conversion function fails to convert the entire field", we are requiring every error condition to store.

Bill: When did this happen?

Alisdair: One of the last two or three meetings.

Dietmar: To store a value in case of failure is a very bad idea.

Move to Open, needs more study.

[2011-03-24 Madrid meeting]

Move to deferred

[2011 Bloomington]

The proposed wording looks good, no-one sure why this was held back before. Move to Review.

[2012,Kona]

Move to Open.

The issues is what to do with -1. Should it match 'C' or do the "sane" thing. A fix here changes behavior, but is probably what we want.

Pablo to provide wording, with help from Howard.

[2015-05-06 Lenexa: Move to Ready]

STL: I like that this uses strtod, which I think is new in C99. that avoids truncation from using atof. I have another issue ...

MC: yes LWG 2403 (stof should call strtod)

PJP: the last line is horrible, you don't assign to err, you call setstate(ios_base::failbit). Ah, no, this is inside num_get so the caller does the setstate.

MC: we need all these words. are they the right words?

JW: I'd like to take a minute to check my impl. Technically this implies a change in behaviour (from always using strtold and checking the extracted floating point value, to using the right function). Oh, we already do exactly this.

MC: Move to Ready

6 in favor, none opposed, 1 abstention

Proposed resolution:

Change 22.4.2.1.2 [facet.num.get.virtuals] as follows:

Stage 3: The sequence of chars accumulated in stage 2 (the field) is converted to a numeric value by the rules of one of the functions declared in the header <cstdlib>:

- For a signed integer value, the function strtoll.
- For an unsigned integer value, the function strtoull.
- For a float value, the function strtod.
- For a double value, the function strtod.
- For a floating-point long double value, the function strtold.

The numeric value to be stored can be one of:

- zero, if the conversion function fails to convert the entire field.
- the most positive (or negative) representable value, if the field to be converted to a signed integer type represents a value too large positive (or negative) to be represented in val.
- the most negative representable value or zero for an unsigned integer type, if the field represents a value too large negative to be represented in val.
- the most positive representable value, if the field to be converted to an unsigned integer type represents a value that cannot be represented in val.
- the converted value, otherwise.

The resultant numeric value is stored in `val`. If the conversion function fails to convert the entire field, or if the field represents a value outside the range of representable values, `ios_base::failbit` is assigned to `err`.

2072. Unclear wording about capacity of temporary buffers

Section: 20.7.11 [temporary.buffer] **Status:** [Ready](#) **Submitter:** Kazutoshi Satoda **Opened:** 2011-08-10 **Last modified:** 2015-05-08

View all other [issues](#) in [temporary.buffer].

View all issues with [Ready](#) status.

Discussion:

According to 20.7.11 [temporary.buffer] p1+2:

```
template <class T>
pair<T*, ptrdiff_t> get_temporary_buffer(ptrdiff_t n) noexcept;
```

-1- *Effects:* Obtains a pointer to storage sufficient to store up to n adjacent T objects. It is implementation-defined whether over-aligned types are supported (3.11).

-2- *Returns:* A pair containing the buffer's address and capacity (in the units of `sizeof(T)`), or a pair of 0 values if no storage can be obtained or if $n \leq 0$.

I read this as prohibiting to return a buffer of which capacity is less than n , because such a buffer is not sufficient to store n objects.

The corresponding description in [SGI STL](#) is clear on this point, but I think it is a bit too verbose:

(for the return value, a pair P) [...] the buffer pointed to by $P.first$ is large enough to hold $P.second$ objects of type T . $P.second$ is greater than or equal to 0, and less than or equal to len .

There seems to be two different targets of the "up to n " modification: The capacity of obtained buffer, and the actual number that the caller will store into the buffer.

First I read as the latter, and got surprised seeing that `libstdc++` implementation can return a smaller buffer. I started searching about `get_temporary_buffer()`. After reading a quote from TC++PL at [stackoverflow](#), I realized that the former is intended.

Such misinterpretation seems common:

- The above question is likely started from same misinterpretation.
- JIS standard (Japanese translation of ISO/IEC standard) says nothing like "up to". I think the editor misinterpreted the original wording, and omitted words for "up to" as it is redundant. (If a buffer is sufficient to store n objects, it is also sufficient to store up to n objects.)
- Rogue Wave implementation doesn't return smaller buffer, instead, it can return larger buffer on some circumstances. Apache [STDCXX](#) is a derived version of that implementation, and [publicly accessible](#):

Specializations of the `get_temporary_buffer()` function template attempt to allocate a region of storage sufficiently large to store at least n adjacent objects of type T .

I know one commercial compiler package based on Rogue Wave implementation, and its implementation is essentially same as the above.

[2014-05-18, Daniel comments and suggests concrete wording]

The provided wording attempts to clarify the discussed capacity freedom, but it also makes it clearer that the returned memory is just "raw memory", which is currently not really clear. In addition the wording clarifies that the deallocating `return_temporary_buffer` function does not throw exceptions, which I believe is the intention when the preconditions of the functions are satisfied. Then, my understanding is that we can provide to `return_temporary_buffer` a null pointer value if that was the value, `get_temporary_buffer()` had returned. Furthermore, as STL noticed, the current wording seemingly allows multiple invocations of `return_temporary_buffer` with the same value returned by `get_temporary_buffer`; this should be constrained similar to the wording we have for `operator delete` (unfortunately we miss such wording for allocators).

MC: move to ready? in favor: 14, opposed: 0, abstain: 0

Proposed resolution:

This wording is relative to N3936.

1. Change 20.7.11 [temporary.buffer] as indicated:

```
template <class T>
pair<T*, ptrdiff_t> get_temporary_buffer(ptrdiff_t n) noexcept;
```

-1- *Effects*: Obtains a pointer to uninitialized, contiguous storage for N adjacent objects of type T , for some non-negative number N . ~~Obtains a pointer to storage sufficient to store up to n adjacent T objects.~~ It is implementation-defined whether over-aligned types are supported (3.11).

-?- *Remarks*: Calling `get_temporary_buffer` with a positive number n is a non-binding request to return storage for n objects of type T . In this case, an implementation is permitted to return instead storage for a non-negative number N of such objects, where $N \neq n$ (including $N = 0$). [Note: The request is non-binding to allow latitude for implementation-specific optimizations of its memory management. — end note].

-2- *Returns*: If $n \leq 0$ or if no storage could be obtained, returns a pair P such that $P.first$ is a null pointer value and $P.second == 0$; otherwise returns a pair P such that $P.first$ refers to the address of the uninitialized storage and $P.second$ refers to its capacity N (in the units of `sizeof(T)`). ~~A pair containing the buffer's address and capacity (in the units of `sizeof(T)`), or a pair of 0 values if no storage can be obtained or if $n \leq -0$.~~

```
template <class T> void return_temporary_buffer(T* p);
```

-3- *Effects*: Deallocates the ~~buffer to which p points~~ storage referenced by p .

-4- *Requires*: ~~The buffer shall have been previously allocated by~~ p shall be a pointer value returned by an earlier call to `get_temporary_buffer` which has not been invalidated by an intervening call to `return_temporary_buffer(T*)`.

-?- *Throws*: Nothing.

2101. Some transformation types can produce impossible types

Section: 20.10.7 [meta.trans] **Status:** [Ready](#) **Submitter:** Daniel Krüglér **Opened:** 2011-11-18 **Last modified:** 2015-05-08

View all issues with [Ready](#) status.

Discussion:

Table 53 — "Reference modifications" says in regard to the type trait `add_lvalue_reference` (emphasize mine)

If T names an object or **function** type then the member typedef type shall name $T\&$;

The problem with this specification is that function types with cv-qualifier or *ref*-qualifier, like

```
void() const
void() &
```

are also affected by the first part of the rule, but this would essentially mean, that instantiating `add_lvalue_reference` with such a type would attempt to form a type that is not defined in the C++ type system, namely

```
void(&()) const
void(&()) &
```

The general policy for *TransformationTraits* is to define always some meaningful mapping type, but this does not hold for `add_lvalue_reference`, `add_rvalue_reference`, and in addition to these two for `add_pointer` as well. The latter one would attempt to form the invalid types

```
void(*)() const
void(*)() &
```

A possible reason why those traits were specified in this way is that in C++03 (and that means for TR1), cv-qualifier were

underspecified in the core language and several compilers just ignored them during template instantiations. This situation became fixed by adopting CWG issues [295](#) and [547](#).

While there is possibly some core language clarification needed (see reflector messages starting from c++std-core-20740), it seems also clear that the library should fix the specification. The suggested resolution follows the style of the specification of the support concepts `PointeeType` and `ReferentType` defined in [N2914](#).

[2012-02-10, Kona]

Move to NAD.

These cv- and ref-qualified function types are aberrations in the type system, and do not represent any actual entity defined by the language. The notion of cv- and ref- qualification applies only to the implicit `*this` reference in a member function.

However, these types can be produced by quirks of template metaprogramming, the question remains what the library should do about it. For example, `add_reference` returns the original type if passed a reference type, or a void type. Conversely, `add_pointer` will return a pointer to the referenced type when passed a reference.

It is most likely that the 'right' answer in any case will depend on the context that the question is being asked, in terms of forming these obscure types. The best the LWG can do is allow an error to propagate back to the user, so they can provide their own meaningful answer in their context - with additional metaprogramming on their part. The consensus is that if anyone is dangerous enough with templates to get themselves into this problem, they will also have the skills to resolve the problem themselves. This is not going to trip up the non-expert developer.

Lastly, it was noted that this problem arises only because the language is inconsistent in providing us these nonsense types that do not really represent anything in the language. There may be some way Core or Evolution could give us a more consistent type system so that the LWG does not need to invent an answer at all, should this question need resolving. This is another reason to not specify anything at the LWG trait level at this time, leaving the other working groups free to produce the 'right' answer that we can then follow without changing the meaning of existing, well-defined programs.

[2012-02-10, post-Kona]

Move back to Open. Daniel is concerned that this is not an issue we can simply ignore, further details to follow.

[2012-10-06, Daniel comments]

This issue really should be resolved as a defect: First, the argument that "forming these obscure types" should "allow an error to propagate" is inconsistent with the exact same "obscure type" that would be formed when `std::add_lvalue_reference<void>` wouldn't have an extra rule for void types, which also cannot form references. The originally proposed resolution attempts to apply the same solution for the same common property of void types and function types with cv-qualifiers or ref-qualifier. These functions had the property of `ReferentType` during concept time (see [CWG 749](#) bullet three for the final wording).

Core issue [CWG 1417](#) has clarified that any attempt to form a reference of a pointer to a function type with cv-qualifiers or ref-qualifier is ill-formed. Unfortunately, many compilers don't implement this yet.

I also would like to warn about so-called "obscure" types: The problem is that these can occur as the side effect of finding a best match overload of function templates, where this type is exactly correct for one of these overloads, but causes a deep (not-sfinae-friendly) error for others where one of these traits are part of the signature.

Existing experience with void types shows, that this extra rule is not so unexpected. Further, any usage of the result types of these traits as argument types or return types of functions would make these ill-formed (and in a template context would be sfinaed away), so the expected effects are rarely unnoticed. Checking all existing explicit usages of the traits `add_rvalue_reference`, `add_lvalue_reference`, and `add_pointer` didn't show any example where the error would be silent: `add_rvalue_reference` is used to specify the return value of `declval()` and the instantiation of `declval<void() const>()` would be invalid, because of the attempt to return a function type. Similarly, `add_lvalue_reference` is used to specify the return type of `unique_ptr<T>::operator*()`. Again, any instantiation with `void() const` wouldn't remain unnoticed. The trait `add_pointer` is used to specify the trait `std::decay` and this is an interesting example, because it is well-formed when instantiated with void types, too, and is heavily used throughout the library specification. All use-cases would not be negatively affected by the suggested acceptance of function types with cv-qualifiers or ref-qualifier, because they involve types that are either function arguments, function parameters or types where references are formed from.

The alternative would be to add an additional extra rule that doesn't define a type member 'type' when we have a function type with cv-qualifiers or ref-qualifier. This is better than the current state but it is not superior than the proposal to specify the result as the original type, because both variants are sfinae-friendly. A further disadvantage of the "non-type" approach here would be that any usage of `std::decay` would require special protection against these function types, because instantiating `std::decay<void() const>` again would lead to a deep, sfinae-unfriendly error.

The following example demonstrates the problem: Even though the second `f` template is the best final match here, the first one will be instantiated. During that process `std::decay<T>::type` becomes instantiated as well and will raise a deep error, because as part of the implementation the trait `std::add_pointer<void() const>` becomes instantiated:

```
#include <type_traits>

template<class T>
typename std::decay<T>::type f(T&& t);

template<class T, class U>
U f(U u);

int main() {
    f<void() const>(0);
}
```

When the here proposed resolution would be applied this program would be well-formed and selects the expected function.

Previous resolution from Daniel [SUPERSEDED]:

- 1. Change Table 53 — "Reference modifications" in 20.10.7.2 [meta.trans.ref] as indicated:

Table 53 — Reference modifications	
Template	Comments
...	
template <class T> struct add_lvalue_reference;	If τ names an object type or if τ names a function type that does not have cv-qualifiers or a ref-qualifier then the member typedef type shall name $\tau\&$; otherwise, if τ names a type "rvalue reference to τ_1 " then the member typedef type shall name $\tau_1\&$; otherwise, type shall name τ .
template <class T> struct add_rvalue_reference;	If τ names an object type or if τ names a function type that does not have cv-qualifiers or a ref-qualifier then the member typedef type shall name $\tau\&\&$; otherwise, type shall name τ . [Note: This rule reflects the semantics of reference collapsing (8.3.2 [dcl.ref]). For example, when a type τ names a type $\tau_1\&$, the type <code>add_rvalue_reference<T>::type</code> is not an rvalue reference. — end note]

- 2. Change Table 56 — "Pointer modifications" in 20.10.7.5 [meta.trans.ptr] as indicated:

Table 56 — Pointer modifications	
Template	Comments
...	
template <class T> struct add_pointer;	The member typedef type shall name the same type as If τ names a function type that has cv-qualifiers or a ref-qualifier then the member typedef type shall name τ; otherwise, it shall name the same type as <code>remove_reference<T>::type*</code> .

The following revised proposed resolution defines - in the absence of a proper core language definition - a new term *referenceable type* as also suggested by the resolution for LWG 2196 as an umbrella of the negation of `void` types and function types with *cv-qualifiers* or *ref-qualifier*. This simplifies and minimizes the requires wording changes.

[2013-09-26, Daniel synchronizes wording with recent draft]

[2014-05-18, Daniel synchronizes wording with recent draft and comments]

My impression is that this urgency of action this issue attempts to point out is partly caused by the fact that even for the most recent C++14 compilers the implementations have just recently changed to adopt the core wording. Examples for these are bug reports to [gcc](#) or [clang](#).

Occasionally the argument has been presented to me that the suggested changes to the traits affected by this issue would lead to irregularities compared to other traits, especially the lack of guarantee that `add_pointer` might not return a pointer or that `add(l/r)value_reference` might not return a reference type. I would like to point out that this kind of divergence is actually already present in most `add/remove` traits: For example, we have no guarantee that `add_const` returns a `const` type (Reference types or function types get special treatments), or that `add_rvalue_reference` returns an rvalue-reference (e.g. when applied to an lvalue-reference type).

Zhihao Yuan brought to my attention, that the originally proposing paper [N1345](#) carefully discussed these design choices.

[2015-05, Lenexa]

MC: move to Ready: in favor: 16, opposed: 0, abstain: 1
STL: have libstdc++, libc++ implemented this? we would need to change your implementation

Proposed resolution:

This wording is relative to N3936.

- 1. Change Table 53 — "Reference modifications" in 20.10.7.2 [meta.trans.ref] as indicated:

Table 53 — Reference modifications	
Template	Comments
...	
template <class T> struct add_lvalue_reference;	If τ names an object or function type a referenceable type then the member typedef type shall name $\tau\&$; otherwise, if τ names a type "rvalue reference to τ_1" then the member typedef type shall name $\tau_1\&$; otherwise, type shall name τ. [Note: This rule reflects the semantics of reference collapsing (8.3.2 [dcl.ref]). — end note]
template <class T> struct add_rvalue_reference;	If τ names an object or function type a referenceable type then the member typedef type shall name $\tau\&\&$; otherwise, type shall name τ . [Note: This rule reflects the semantics of reference collapsing (8.3.2 [dcl.ref]). For example, when a type τ names a type $\tau_1\&$, the type <code>add_rvalue_reference_t<T></code> is not an rvalue reference. — end note]

- 2. Change Table 56 — "Pointer modifications" in 20.10.7.5 [meta.trans.ptr] as indicated:

Table 56 — Pointer modifications	
Template	Comments
...	
template <class T> struct add_pointer; remove_reference_t<T>*	If τ names a referenceable type or a (possibly cv-qualified) void type then The member typedef type shall name the same type as otherwise, type shall name τ.

[2111](#). Which unexpected/terminate handler is called from the exception handling runtime?

Section: 18.8.3.4 [terminate], D.8.4 [unexpected] **Status:** [Ready](#) **Submitter:** Howard Hinnant **Opened:** 2011-12-06 **Last modified:** 2015-09-25

View all other [issues](#) in [terminate].

View all issues with [Ready](#) status.

Discussion:

Prior to N3242, modified by N3189, we said this about `unexpected()`:

Effects: Calls the `unexpected_handler` function in effect immediately after evaluating the throw-expression (D.13.1), if called by the implementation, or calls the current `unexpected_handler`, if called by the program.

and this about `terminate()`:

Effects: Calls the `terminate_handler` function in effect immediately after evaluating the throw-expression (18.8.3.1), if called by the implementation, or calls the current `terminate_handler` function, if called by the program.

But now in both places we say:

Calls the current `unexpected_handler` function.

and:

Calls the current `terminate` function.

The difference is that in C++98/03 if a destructor reset a handler during stack unwinding, that new handler was not called if the unwinding later led to `unexpected()` or `terminate()` being called. But these new words say that this new handler *is* called. This is an ABI-breaking change in the way exceptions are handled. Was this change intentional?

N3189 was mainly about introducing exception safety and getters for the handlers. I don't recall the issue of *which* handler gets called being part of the discussion.

I propose that we revert to the C++98/03 behavior in this regard, lest ABI's such as the Itanium ABI are invalidated. A mechanical way to do this is to revert bullets 9 and 12 of N3189.

[2011-12-09: Daniel comments]

There was no such semantic change intended. It was an unfortunate side effect when trying to better separate different responsibilities in the previous wording.

A related issue is [2088](#).

[2012-01-30: Howard comments]

The C++98/03 wording is somewhat ambiguous:

Calls the `terminate_handler` function in effect immediately after evaluating the throw-expression...

There are potentially two throw-expressions being referred to here, and it is not clear if this sentence is referring to just the first or both:

1. `throw assignment-expression;`
2. `throw;`

There is ample evidence in current implementations that it is understood that *only* 1. was meant. But clearly both 1 and 2 could have been meant. We need a clarification. Does an execution of a rethrow (`throw;`) update which handlers can potentially be called?

2. `throw;` // update handlers to `get_XXX()`?

My opinion: Go with existing practice, and clarify what that practice is, if surveys find that everyone does the same thing. Gcc 4.2 and Apple do 1. only, and do not reset the handlers to the current handlers on `throw;`.

If current practice is not unanimously one way or the other, I have no strong opinion. I have not found a motivating use case for the use of any particular handler. Most applications set the handlers once at the beginning of the program and then do not change them, and so will not be impacted by whatever decision is made here.

[2014-02-15 Issaquah: Move to Review]

STL: Original change in N3242 came from trying to make `set/get` exception handler thread safe. The issue requests we revert to 98/03, which Howard notes was already ambiguous.

Alisdair: Issue author thinks we made this change in C++11 without taking into account Itanium ABI, which cannot implement the new semantic (without breaking compatibility).

Alisdair: original change in N3242 was trying to solve the problem of which handler is called when the handler is changing in another thread, but this turns out to be an issue in even the single-threaded case.

Pablo: despite wanting to make it thread safe, you are still changing a global

STL and Marshall confirm that there is real implementation divergence on the question, so we cannot pick just one behavior if we want to avoid breaking existing practice.

Alisdair: not sure who to talk to across all library vendors to fix, need more information for progress (IBM and Sun)

STL: Howard did identify a problem with the wording as well: `throw;` is a throw expression, but we typically want to re-activate the in-flight exception, not throw a new copy.

Pablo: wondering why all of this wording is here (N3189)? It looks like we were trying to handle another thread changing handler between a `throw` and `terminate` in current thread.

Alisdair: Anything working with exception handling should have used only thread-local resources, but that ship has sailed. We must account for the same exception object being re-thrown in multiple threads simultaneously, with no happens-before relationships.

Room: Why on earth would we care about exactly which way the program dies when the terminate calls are racing?!

Pablo: Reasonable to set the handler once (in `main`) and never change it.

Pablo: If willing to put lots of work into this, you could say at point of a `throw` these handlers become thread local but that is overkill. We want destructors to be able to change these handlers (if only for backwards compatibility).

Alisdair: the "*do it right*" is to do something per thread, but that is more work than vendors will want to do. Want to say setting handler while running multiple threads is unspecified.

Pablo: possible all we need to do is say it is always the current handler

STL: That prevents an implementation or single threaded program from calling a new handler after a `throw`, probably should say if `terminate` is called *by the implementation* (during EH), any handler that was current can be called. Leaves it up in the air as to when the handler is captured, supporting the diverging existing practices.

Jeffrey: use *happens before* terminology to avoid introducing races

STL: Give this to concurrency?

Jeffrey: It is in clause 18, generally LWG and not SG1 territory.

Alisdair: Concerned about introducing *happens before* into fundamental exception handling since it would affect single threaded performance as well. Want to give to concurrency or LEWG/EWG, we are into language design here.

Jeffrey: suspect LEWG won't have a strong opinion. I don't want it to be ours!!!

Pablo: Might be a case for core>

Alisdair: Would be happier if at least one core person were in the discussion.

STL: No sympathy for code that tries to guard the terminate handler.

Alisdair: We are back to set it once, globally. Want to be clear that if `set_terminate` is called just once, when EH is not active, and never changed again, then the user should get the handler from that specific call.

AlisdairM: "unspecified which handler is called if an exception is active when `set_terminate` is called." This supports existing behaviors, and guarantees which handler is called in non-contentious situations. Implicit assumption that a function becomes a handler only after a successful call to `set_handler`, so we are not leaving a door open to the implementation inventing entirely new handlers of its own.

Consensus.

Poll to confirm status as P1: new consensus is P3

Action: Alisdair provides new wording. Drop from P1 to P3, and move to Review.

[2015-05, Lenexa]

HH: we accidentally changed semantics of which handler gets called during exception unwinding. This was attempt to put it back. Discovered implementations don't really do anything. [...] Fine with unspecified behavior to move this week.

STL/MC: observed different behavior

STL: legitimizes all implementations and tells users to not do this

Move to ready? 9/0/1

Proposed resolution:

Amend 18.8.3.4 [terminate] as indicated:

```
[[noreturn]] void terminate() noexcept;
```

Remarks: Called by the implementation when exception handling must be abandoned for any of several reasons (15.5.1), ~~in effect immediately after throwing the exception~~. May also be called directly by the program.

Effects: Calls a terminate handler function. It is unspecified which terminate handler function will be called if an exception is active during a call to `set_terminate`. Otherwise c~~alls~~ the current `terminate_handler` function. [Note: A default `terminate_handler` is always considered a callable handler in this context. — *end note*]

Amend D.8.4 [unexpected] as indicated:

```
[[noreturn]] void unexpected();
```

Remarks: Called by the implementation when a function exits via an exception not allowed by its *exception-specification* (15.5.2), ~~in effect after evaluating the throw-expression (D.11.1)~~. May also be called directly by the program.

Effects: Calls an unexpected handler function. It is unspecified which unexpected handler function will be called if an exception is active during a call to set_unexpected. Otherwise c~~alls~~ the current unexpected_handler function. [Note: A default unexpected_handler is always considered a callable handler in this context. — *end note*]

2119. Missing hash specializations for extended integer types

Section: 20.9.13 [unord.hash] **Status:** [Ready](#) **Submitter:** Daniel Krügler **Opened:** 2011-12-16 **Last modified:** 2015-05-08

View other [active issues](#) in [unord.hash].

View all other [issues](#) in [unord.hash].

View all issues with [Ready](#) status.

Discussion:

According to the header <functional> synopsis 20.9 [function.objects] and to the explicit description in 20.9.13 [unord.hash] class template hash specializations shall be provided for all arithmetic types that are not extended integer types. This is not explicitly mentioned, but neither the list nor any normative wording does include them, so it follows by implication.

What are the reasons that extended integer types are excluded? E.g. for numeric_limits corresponding specializations are required. I would expect that an unordered_map with key type std::uintmax_t would just work, but that depends now on whether this type is an extended integer type or not.

This issue is *not* asking for also providing specializations for the cv-qualified arithmetic types. While this is surely a nice-to-have feature, I consider that restriction as a more secondary problem in practice.

The proposed resolution also fixes a problem mentioned in [2109](#) in regard to confusing requirements on user-defined types and those on implementations.

[2012, Kona]

Move to Open.

Agreed that it's a real issue and that the proposed wording fixes it. However, the wording change is not minimal and isn't consistent with the way we fixed hash wording elsewhere.

Alisdair will provide updated wording.

[2014-05-06 Geoffrey Romer suggests alternative wording]

Previous resolution from Daniel [SUPERSEDED]:

This wording is relative to the FDIS.

Change 20.9.13 [unord.hash] p2 as indicated:

```
template <> struct hash<bool>;
template <> struct hash<char>;
[...]
template <> struct hash<long double>;
template <class T> struct hash<T*>;
```

~~-2- Requires: the template specializations shall meet the requirements of class template hash (20.9.13 [unord.hash])~~ The header <functional> provides definitions for specializations of the hash class template for each cv-unqualified arithmetic type. This header also provides a definition for a partial specialization of the hash class template for any pointer type. The requirements for the members of these specializations are given in sub-clause 20.9.13 [unord.hash].

[2015-05, Lenexa]

STL: the new PR is very simple and could resolve that nicely

MC: the older PR is rather longish

- anybody have any objections to this approach?
- what people want to have as a status?

STL: I want to have Ready

MC: move to ready: in favor: 13, opposed: 0, abstain: 4

Proposed resolution:

This wording is relative to N3936.

Change 20.9.13 [unord.hash] p1 as indicated:

The unordered associative containers defined in 23.5 use specializations of the class template `hash` as the default hash function. For all object types `Key` for which there exists a specialization `hash<Key>`, and for all **integral and** enumeration types (7.2) `Key`, the instantiation `hash<Key>` shall: [...]

[2127](#). Move-construction with `raw_storage_iterator`

Section: 20.7.10 [storage.iterator] **Status:** [Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2012-01-23 **Last modified:** 2015-05-08

View all other [issues](#) in [storage.iterator].

View all issues with [Ready](#) status.

Discussion:

Aliaksandr Valialkin pointed out that `raw_storage_iterator` only supports constructing a new object from lvalues so cannot be used to construct move-only types:

```
template <typename InputIterator, typename T>
void move_to_raw_buffer(InputIterator first, InputIterator last, T *raw_buffer)
{
    std::move(first, last, std::raw_storage_iterator<T *, T>(raw_buffer));
}
```

This could easily be solved by overloading `operator=` for rvalues.

Dave Abrahams:

`raw_storage_iterator` causes exception-safety problems when used with any generic algorithm. I suggest leaving it alone and not encouraging its use.

[2014-11-11, Jonathan provides improved wording]

In Urbana LWG decided to explicitly say the value is constructed from an rvalue.

Previous resolution from Jonathan [SUPERSEDED]:

This wording is relative to [N3337](#).

1. Add a new signature to the synopsis in 20.7.10 [storage.iterator] p1:

```
namespace std {
    template <class OutputIterator, class T>
    class raw_storage_iterator
        : public iterator<output_iterator_tag,void,void,void,void> {
    public:
        explicit raw_storage_iterator(OutputIterator x);

        raw_storage_iterator<OutputIterator,T>& operator*();
        raw_storage_iterator<OutputIterator,T>& operator=(const T& element);
        raw_storage_iterator<OutputIterator,T>& operator=(T&& element);
        raw_storage_iterator<OutputIterator,T>& operator++();
        raw_storage_iterator<OutputIterator,T> operator++(int);
    };
}
```

2. Insert the new signature and a new paragraph before p4:

```
raw_storage_iterator<OutputIterator,T>& operator=(const T& element);  
raw_storage_iterator<OutputIterator,T>& operator=(T&& element);
```

-?- Requires: For the first signature T shall be CopyConstructible. For the second signature T shall be MoveConstructible.

-4- Effects: Constructs a value from `element` at the location to which the iterator points.

-5- Returns: A reference to the iterator.

[2015-05, Lenexa]

MC: Suggestion to move it to Ready for incorporation on Friday

MC: move to ready: in favor: 12, opposed: 0, abstain: 3

Proposed resolution:

This wording is relative to [N4140](#).

1. Add a new signature to the synopsis in 20.7.10 [storage.iterator] p1:

```
namespace std {  
    template <class OutputIterator, class T>  
    class raw_storage_iterator  
        : public iterator<output_iterator_tag,void,void,void,void> {  
    public:  
        explicit raw_storage_iterator(OutputIterator x);  
  
        raw_storage_iterator<OutputIterator,T>& operator*();  
        raw_storage_iterator<OutputIterator,T>& operator=(const T& element);  
        raw_storage_iterator<OutputIterator,T>& operator=(T&& element);  
        raw_storage_iterator<OutputIterator,T>& operator++();  
        raw_storage_iterator<OutputIterator,T> operator++(int);  
};  
}
```

2. Insert a new paragraph before p4:

```
raw_storage_iterator<OutputIterator,T>& operator=(const T& element);
```

-?- Requires: T shall be CopyConstructible.

-4- Effects: Constructs a value from `element` at the location to which the iterator points.

-5- Returns: A reference to the iterator.

3. Insert the new signature and a new paragraph after p5:

```
raw_storage_iterator<OutputIterator,T>& operator=(T&& element);
```

-?- Requires: T shall be MoveConstructible.

-?- Effects: Constructs a value from `std::move(element)` at the location to which the iterator points.

-?- Returns: A reference to the iterator.

2133. Attitude to overloaded comma for iterators

Section: 17.6.5.4 [global.functions] **Status:** [Ready](#) **Submitter:** Yakov Galka **Opened:** 2012-01-25 **Last modified:** 2015-05-08

View all other [issues](#) in [global.functions].

View all issues with [Ready](#) status.

Discussion:

17.6.5.4 [global.functions] says

Unless otherwise specified, global and non-member functions in the standard library shall not use functions from another namespace which are found through argument-dependent name lookup (3.4.2).

This sounds clear enough. There are just two problems:

1. Both implementations I tested (VS2005 and GCC 3.4.3) do unqualified calls to the comma operator in some parts of the library with operands of user-defined types.
2. The standard itself does this in the description of some algorithms. E.g. `uninitialized_copy` is defined as:

Effects:

```
for (; first != last; ++result, ++first)
    ::new (static_cast<void*>(&*result))
        typename iterator_traits<ForwardIterator>::value_type(*first);
```

If understood literally, it is required to call `operator,(ForwardIterator, InputIterator)`.

For detailed discussion with code samples see [here](#).

Proposal:

1. Add an exception to the rule in 17.6.5.4 [global.functions] by permitting the implementation to call the comma operator as much as it wants to. I doubt we want this. or
2. Fix the description of the said algorithms and perhaps add a note to 17.6.5.4 [global.functions] that brings attention of the implementers to avoid this pitfall.

[2013-03-15 Issues Teleconference]

Moved to Open.

There are real questions here, that may require a paper to explore and answer properly.

[2014-05-18, Daniel comments and suggests concrete wording]

Other issues, such as [2114](#) already follow a similar spirit as the one suggested by bullet 2 of the issue submitter. I assert that consideration of possible user-provided overloads of the comma-operator were not intended by the original wording and doing so afterwards would unnecessarily complicate a future conceptualization of the library and would needlessly restrict implementations.

I don't think that a paper is needed to solve this issue, there is a simply way to ensure that the code-semantics excludes consideration of user-provided comma operators. The provided wording below clarifies this by explicitly casting the first argument of the operator to `void`.

[2015-05, Lenexa]

DK: is putting it in the middle the right place for it?

STL: either works, but visually putting it in the middle is clearer, and for `"++it1, ++i2, ++it3"` it needs to be done after the second comma, so `"++it1, (void) ++i2, (void) ++it3"` is better than `"(void) ++it1, ++i2, (void) ++it3"`

ZY: for *INVOKE* yesterday we used `static_cast<void>` but here we're using C-style cast, why?

STL: for *INVOKE* I want to draw attention that there's an intentional coercion to `void` because that's the desired type. Here we only do it because that's the best way to prevent the problem, not because we specifically want a `void` type.

Move to Ready: 9 in favor, none opposed, 1 abstention

Proposed resolution:

This wording is relative to N3936.

1. Change 20.7.12.2 [uninitialized.copy] as indicated:

```
template <class InputIterator, class ForwardIterator>
    ForwardIterator uninitialized_copy(InputIterator first, InputIterator last,
                                     ForwardIterator result);
```

-1- Effects:

```
for (; first != last; ++result, (void) ++first)
    ::new (static_cast<void*>(&*result))
        typename iterator_traits<ForwardIterator>::value_type(*first);
```

[...]

```
template <class InputIterator, class Size, class ForwardIterator>
    ForwardIterator uninitialized_copy_n(InputIterator first, Size n,
```

```
ForwardIterator result);
```

-3- Effects:

```
for (; n > 0; ++result, (void) ++first, --n)
    ::new (static_cast<void*>(&*result))
        typename iterator_traits<ForwardIterator>::value_type(*first);
```

2. Change 25.4.8 [alg.lex.comparison] p3 as indicated:

```
template<class InputIterator1, class InputIterator2>
bool
lexicographical_compare(InputIterator1 first1, InputIterator1 last1,
                        InputIterator2 first2, InputIterator2 last2);
template<class InputIterator1, class InputIterator2, class Compare>
bool
lexicographical_compare(InputIterator1 first1, InputIterator1 last1,
                        InputIterator2 first2, InputIterator2 last2,
                        Compare comp);
```

-3- Remarks: [...]

```
for (; first1 != last1 && first2 != last2; ++first1, (void) ++first2) {
    if (*first1 < *first2) return true;
    if (*first2 < *first1) return false;
}
return first1 == last1 && first2 != last2;
```

2156. Unordered containers' `reserve(n)` reserves for $n-1$ elements

Section: 23.2.5 [unord.req] **Status:** [Ready](#) **Submitter:** Daniel James **Opened:** 2012-05-07 **Last modified:** 2015-05-22

View other [active issues](#) in [unord.req].

View all other [issues](#) in [unord.req].

View all issues with [Ready](#) status.

Discussion:

I think that unordered containers' `reserve` doesn't quite do what it should. I'd expect after calling `x.reserve(n)` to be able to insert n elements without invalidating iterators. But as the standard is written (I'm looking at n3376), I think the guarantee only holds for $n-1$ elements.

For a container with `max_load_factor` of 1, `reserve(n)` is equivalent to `rehash(ceil(n/1))`, i.e. `rehash(n)`. `rehash(n)` requires that the bucket count is $\geq n$, so it can be n (Table 103). The rule is that `insert` shall not affect the validity of iterators if $(N + n) < z * B$ (23.2.5 [unord.req] p15). But for this case the two sides of the equation are equal, so `insert` can affect the validity of iterators.

[2013-03-16 Howard comments and provides wording]

Given the following:

```
LF := load_factor()
MLF := max_load_factor()
S := size()
B := bucket_count()

LF == S/B
```

The container has an invariant:

```
LF <= MLF
```

Therefore:

```
MLF >= S/B
S <= MLF * B
B >= S/MLF
```

[2013-03-15 Issues Teleconference]

Moved to Open.

Howard to provide rationale and potentially revised wording.

[2012-02-12 Issaquah : recategorize as P3]

Jonathon Wakely: submitter is Boost.Hash maintainer. Think it's right.

Marshall Clow: even if wrong it's more right than what we have now

Geoffrey Romer: issue is saying rehash should not leave container in such a state that a notional insertion of zero elements should not trigger a rehash

AJM: e.g. if you do a range insert from an empty range

AJM: we don't have enough brainpower to do this now, so not priority zero

Recategorised as P3

[Lenexa 2015-05-06: Move to Ready]

Proposed resolution:

This wording is relative to N3485.

1. In 23.2.5 [unord.req] Table 103 — Unordered associative container requirements, change the post-condition in the row for `a.rehash(n)` to:

Post: `a.bucket_count() >= a.size() / a.max_load_factor()` and `a.bucket_count() >= n`.

2. In 23.2.5 [unord.req]/p15 change

The `insert` and `emplace` members shall not affect the validity of iterators if $(N+n) < z * B$, where N is the number of elements in the container prior to the insert operation, n is the number of elements inserted, B is the container's bucket count, and z is the container's maximum load factor.

2181. Exceptions from *seed sequence* operations

Section: 26.5.1.2 [rand.req.seedseq], 26.5.3 [rand.eng], 26.5.4 [rand.adapt] **Status:** [Ready](#) **Submitter:** Daniel Krüger **Opened:** 2012-08-18 **Last modified:** 2015-09-25

View all other [issues](#) in [rand.req.seedseq].

View all issues with [Ready](#) status.

Discussion:

LWG issue [2180](#) points out some deficiencies in regard to the specification of the library-provided type `std::seed_seq` regarding exceptions, but there is another specification problem in regard to general types satisfying the *seed sequence* constraints (named `sseq`) as described in 26.5.1.2 [rand.req.seedseq].

26.5.3 [rand.eng] p3 and 26.5.4.1 [rand.adapt.general] p3 say upfront:

Except where specified otherwise, no function described in this section 26.5.3 [rand.eng]/26.5.4 [rand.adapt] throws an exception.

This constraint causes problems, because the described templates in these sub-clauses depend on operations of `sseq::generate()` which is a function template, that depends both on operations provided by the implementor of `sseq` (e.g. of `std::seed_seq`), and those of the random access iterator type provided by the caller. With class template `linear_congruential_engine` we have just one example for a user of `sseq::generate()` via:

```
template<class Sseq>
linear_congruential_engine<>::linear_congruential_engine(Sseq& q);

template<class Sseq>
void linear_congruential_engine<>::seed(Sseq& q);
```

None of these operations has an exclusion rule for exceptions.

As described in [2180](#) the wording for `std::seed_seq` should and can be fixed to ensure that operations of `seed_seq::generate()` won't throw except from operations of the provided iterator range, but there is no corresponding "safety belt" for user-provided `sseq` types, since 26.5.1.2 [rand.req.seedseq] does not impose no-throw requirements onto operations of *seed sequences*.

- I. A quite radical step to fix this problem would be to impose general no-throw requirements on the expression `q.generate(rb,re)` from Table 115, but this is not as simple as it looks initially, because this function again depends on general types that are mutable random access iterators. Typically, we do not impose no-throw requirements on iterator operations and this would restrict general seed sequences where exceptions are not a problem. Furthermore, we do not impose comparable constraints for other expressions, like that of the expression `g()` in Table 116 for good reasons, e.g. `random_device::operator()` explicitly states when it throws exceptions.
- II. A less radical variant of the previous suggestion would be to add a normative requirement on the expression `q.generate(rb,re)` from Table 115 that says: "Throws nothing if operations of `rb` and `re` do not throw exceptions". Nevertheless we typically do not describe *conditional* Throws elements in proper requirement sets elsewhere (Container requirements excluded, they just describe the containers from Clause 23) and this may exclude resonable implementations of seed sequences that could throw exceptions under rare situations.
- III. The iterator arguments provided to `SSeq::generate()` for operations in templates of 26.5.3 [rand.eng] and 26.5.4 [rand.adapt] are under control of implementations, so we could impose stricter exceptions requirements on `SSeq::generate()` for `SSeq` types that are used to instantiate member templates in 26.5.3 [rand.eng] and 26.5.4 [rand.adapt] solely.
- IV. We simply add extra wording to the introductive parts of 26.5.3 [rand.eng] and 26.5.4 [rand.adapt] that specify that operations of the engine (adaptor) templates that depend on a template parameter `SSeq` throw no exception unless `SSeq::generate()` throws an exception.

Given these options I would suggest to apply the variant described in the fourth bullet.

The proposed resolution attempts to reduce a lot of the redundancies of requirements in the introductory paragraphs of 26.5.3 [rand.eng] and 26.5.4 [rand.adapt] by introducing a new intermediate sub-clause "Engine and engine adaptor class templates" following sub-clause 26.5.2 [rand.synopsis]. This approach also solves the problem that currently 26.5.3 [rand.eng] also describes requirements that apply for 26.5.4 [rand.adapt] (Constrained templates involving the `SSeq` parameters).

[2013-04-20, Bristol]

Remove the first bullet point:

?- Throughout this sub-clause general requirements and conventions are described that apply to every class template specified in sub-clause 26.5.3 [rand.eng] and 26.5.4 [rand.adapt]. Phrases of the form "in those sub-clauses" shall be interpreted as equivalent to "in sub-clauses 26.5.3 [rand.eng] and 26.5.4 [rand.adapt]".

Replace "in those sub-clauses" with "in sub-clauses 26.5.3 [rand.eng] and 26.5.4 [rand.adapt]".

Find another place for the wording.

Daniel: These are requirements on the implementation not on the types. I'm not comfortable in moving it to another place without double checking.

Improve the text (there are 4 "for"s): *for* copy constructors, *for* copy assignment operators, *for* streaming operators, and *for* equality and inequality operators are not shown in the synopses.

Move the information of this paragraph to the paragraphs it refers to:

"?- Descriptions are provided in those sub-clauses only for engine operations that are not described in 26.5.1.4 [rand.req.eng], for adaptor operations that are not described in 26.5.1.5 [rand.req.adapt], or for operations where there is additional semantic information. In particular, declarations for copy constructors, for copy assignment operators, for streaming operators, and for equality and inequality operators are not shown in the synopses."

Alisdair: I prefer duplication here than consolidation/reference to these paragraphs.

The room showed weakly favjust or for duplication.

Previous resolution from Daniel [SUPERSEDED]:

1. Add a new sub-clause titled "Engine and engine adaptor class templates" following sub-clause 26.5.2 [rand.synopsis] (but at the same level) and add one further sub-clause "General" as child of the new sub-clause as follows:

Engine and engine adaptor class templates [rand.engadapt]

General [rand.engadapt.general]

-?- Throughout this sub-clause general requirements and conventions are described that apply to every class template specified in sub-clause 26.5.3 [rand.eng] and 26.5.4 [rand.adapt]. Phrases of the form "in those sub-clauses" shall be interpreted as equivalent to "in sub-clauses 26.5.3 [rand.eng] and 26.5.4 [rand.adapt]".

-?- Except where specified otherwise, the complexity of each function specified in those sub-clauses is constant.

-?- Except where specified otherwise, no function described in those sub-clauses throws an exception.

-?- Every function described in those sub-clauses that has a function parameter `q` of type `Sseq&` for a template type parameter named `Sseq` that is different from type `std::seed_seq` throws what and when the invocation of `q.generate` throws.

-?- Descriptions are provided in those sub-clauses only for engine operations that are not described in 26.5.1.4 [rand.req.eng], for adaptor operations that are not described in 26.5.1.5 [rand.req.adapt], or for operations where there is additional semantic information. In particular, declarations for copy constructors, for copy assignment operators, for streaming operators, and for equality and inequality operators are not shown in the synopses.

-?- Each template specified in those sub-clauses requires one or more relationships, involving the value(s) of its non-type template parameter(s), to hold. A program instantiating any of these templates is ill-formed if any such required relationship fails to hold.

-?- For every random number engine and for every random number engine adaptor `x` defined in those sub-clauses:

- if the constructor

```
template <class Sseq> explicit X(Sseq& q);
```

is called with a type `Sseq` that does not qualify as a seed sequence, then this constructor shall not participate in overload resolution;

- if the member function

```
template <class Sseq> void seed(Sseq& q);
```

is called with a type `Sseq` that does not qualify as a seed sequence, then this function shall not participate in overload resolution;

The extent to which an implementation determines that a type cannot be a seed sequence is unspecified, except that as a minimum a type shall not qualify as a seed sequence if it is implicitly convertible to `X::result_type`.

2. Edit the contents of sub-clause 26.5.3 [rand.eng] as indicated:

-1- Each type instantiated from a class template specified in this section 26.5.3 [rand.eng] satisfies the requirements of a random number engine (26.5.1.4 [rand.req.eng]) type and the general implementation requirements specified in sub-clause [rand.engadapt.general].

~~-2- Except where specified otherwise, the complexity of each function specified in this section 26.5.3 [rand.eng] is constant.~~

~~-3- Except where specified otherwise, no function described in this section 26.5.3 [rand.eng] throws an exception.~~

~~-4- Descriptions are provided in this section 26.5.3 [rand.eng] only for engine operations that are not described in 26.5.1.4 [rand.req.eng] [...]~~

~~-5- Each template specified in this section 26.5.3 [rand.eng] requires one or more relationships, involving the value(s) of its non-type template parameter(s), to hold. [...]~~

~~-6- For every random number engine and for every random number engine adaptor `x` defined in this subclause (26.5.3 [rand.eng]) and in sub-clause 26.5.3 [rand.eng]: [...]~~

3. Edit the contents of sub-clause 26.5.4.1 [rand.adapt.general] as indicated:

-1- Each type instantiated from a class template specified in this section 26.5.3 [rand.eng] 26.5.4 [rand.adapt] satisfies the requirements of a random number engine adaptor (26.5.1.5 [rand.req.adapt]) type and the general implementation requirements specified in sub-clause [rand.engadapt.general].

-2- Except where specified otherwise, the complexity of each function specified in this section 26.5.4 [rand.adapt] is constant.

-3- Except where specified otherwise, no function described in this section 26.5.4 [rand.adapt] throws an exception.

-4- Descriptions are provided in this section 26.5.4 [rand.adapt] only for engine operations that are not described in 26.5.1.5 [rand.req.adapt] [...]

-5- Each template specified in this section 26.5.4 [rand.adapt] requires one or more relationships, involving the value(s) of its non-type template parameter(s), to hold. [...]

[2014-02-09, Daniel provides alternative resolution]

[Lenexa 2015-05-07: Move to Ready]

LWG 2181 exceptions from seed sequence operations

STL: Daniel explained that I was confused. I said, oh, seed_seq says it can throw if the RanIt throws. Daniel says the RanIts are provided by the engine. Therefore if you give a seed_seq to an engine, it cannot throw, as implied by the current normative text. So what Daniel has in the PR is correct, if slightly unnecessary. It's okay to have explicitly non-overlapping Standardese even if overlapping would be okay.

Marshall: And this is a case where the std:: on seed_seq is a good thing.

STL: Meh.

STL: And that was my only concern with this PR. I like the latest PR much better than the previous.

Marshall: Yes. There's a drive-by fix for referencing the wrong section. Other than that, the two are the same.

STL: Alisdair wanted the repetition instead of centralization, and I agree.

Marshall: Any other opinions?

Jonathan: I'll buy it.

STL: For a dollar?

Hwrd: I'll buy that for a nickel.

Marshall: Any objections to Ready? I don't see a point in Immediate.

Jonathan: Absolutely agree.

Marshall: 7 for ready, 0 opposed, 0 abstain.

[2014-05-22, Daniel syncs with recent WP]

Proposed resolution:

This wording is relative to N3936.

1. Edit the contents of sub-clause 26.5.3 [rand.eng] as indicated:

-1- Each type instantiated from a class template specified in this section 26.5.3 [rand.eng] satisfies the requirements of a random number engine (26.5.1.4 [rand.req.eng]) type.

-2- Except where specified otherwise, the complexity of each function specified in this section 26.5.3 [rand.eng] is constant.

-3- Except where specified otherwise, no function described in this section 26.5.3 [rand.eng] throws an exception.

-?- Every function described in this section 26.5.3 [rand.eng] that has a function parameter q of type Sseq& for a

template type parameter named `Sseq` that is different from type `std::seed_seq` throws what and when the invocation of `q.generate` throws.

-4- Descriptions are provided in this section 26.5.3 [rand.eng] only for engine operations that are not described in 26.5.1.4 [rand.req.eng] or for operations where there is additional semantic information. In particular, declarations for copy constructors, ~~for~~ copy assignment operators, ~~for~~ streaming operators, ~~and for~~ equality operators, and inequality operators are not shown in the synopses.

-5- Each template specified in this section 26.5.3 [rand.eng] requires one or more relationships, involving the value(s) of its non-type template parameter(s), to hold. A program instantiating any of these templates is ill-formed if any such required relationship fails to hold.

-6- For every random number engine and for every random number engine adaptor `x` defined in this subclause (26.5.3 [rand.eng]) and in sub-clause 26.5.4 [rand.adapt]:

- if the constructor

```
template <class Sseq> explicit X(Sseq& q);
```

is called with a type `Sseq` that does not qualify as a seed sequence, then this constructor shall not participate in overload resolution;

- if the member function

```
template <class Sseq> void seed(Sseq& q);
```

is called with a type `Sseq` that does not qualify as a seed sequence, then this function shall not participate in overload resolution;

The extent to which an implementation determines that a type cannot be a seed sequence is unspecified, except that as a minimum a type shall not qualify as a seed sequence if it is implicitly convertible to `X::result_type`.

2. Edit the contents of sub-clause 26.5.4.1 [rand.adapt.general] as indicated:

-1- Each type instantiated from a class template specified in this section 26.5.3 [rand.eng] 26.5.4 [rand.adapt] satisfies the requirements of a random number engine adaptor (26.5.1.5 [rand.req.adapt]) type.

-2- Except where specified otherwise, the complexity of each function specified in this section 26.5.4 [rand.adapt] is constant.

-3- Except where specified otherwise, no function described in this section 26.5.4 [rand.adapt] throws an exception.

-?- Every function described in this section 26.5.4 [rand.adapt] that has a function parameter `q` of type `Sseq&` for a template type parameter named `Sseq` that is different from type `std::seed_seq` throws what and when the invocation of `q.generate` throws.

-4- Descriptions are provided in this section 26.5.4 [rand.adapt] only for adaptor operations that are not described in section 26.5.1.5 [rand.req.adapt] or for operations where there is additional semantic information. In particular, declarations for copy constructors, ~~for~~ copy assignment operators, ~~for~~ streaming operators, ~~and for~~ equality operators, and inequality operators are not shown in the synopses.

-5- Each template specified in this section 26.5.4 [rand.adapt] requires one or more relationships, involving the value(s) of its non-type template parameter(s), to hold. A program instantiating any of these templates is ill-formed if any such required relationship fails to hold.

3. Edit the contents of sub-clause 26.5.8.1 [rand.dist.general] p2 as indicated: [Drafting note: These editorial changes are just for consistency with those applied to 26.5.3 [rand.eng] and 26.5.4.1 [rand.adapt.general] — end drafting note]

-2- Descriptions are provided in this section 26.5.8 [rand.dist] only for distribution operations that are not described in 26.5.1.6 [rand.req.dist] or for operations where there is additional semantic information. In particular, declarations for copy constructors, ~~for~~ copy assignment operators, ~~for~~ streaming operators, ~~and for~~ equality operators, and inequality operators are not shown in the synopses.

2218. Unclear how containers use `allocator_traits::construct()`

modified: 2015-05-08

View other [active issues](#) in [container.requirements.general].

View all other [issues](#) in [container.requirements.general].

View all issues with [Ready](#) status.

Discussion:

Firstly, 23.2.1 [container.requirements.general]/7 says a container's allocator is used to obtain memory, but it isn't stated explicitly that the same allocator is used to construct and destroy elements, as opposed to a value-initialized allocator of the same type.

Secondly, 23.2.1 [container.requirements.general]/3 says elements "shall be constructed using the `allocator_traits<allocator_type>::construct` function and destroyed using the `allocator_traits<allocator_type>::destroy` function" and 23.2.1 [container.requirements.general]/13 defines `CopyInsertable` etc. in terms of an allocator `A` which is identical to the container's `allocator_type`.

The intent of making `construct()` and `destroy()` function templates was that containers would be permitted to use `allocator_traits<A>::construct()` instead of `allocator_traits<allocator_type>::construct()`, where `A` is `allocator_traits<allocator_type>::rebind_alloc<U>` for some other type `U`. This allows node-based containers to store an allocator of the right type for allocating nodes and to use the same object to construct elements in aligned storage within those nodes, avoiding rebinding and copying the stored allocator every time an element needs to be constructed.

It should be made clear that a possibly-rebound copy of the container's allocator is used for object construction.

[2013-03-15 Issues Teleconference]

Moved to Open.

Jonathan: point 2 in the proposed resolution is definitely needed.

[2014-11-28, Jonathan improves wording]

In the first set of edits to paragraph 3 both pieces inserting "`rebind_alloc<U>::`" should be replaced by "`rebind_traits<U>::`"

Otherwise it implies using the allocator's functions directly, but they might not exist and it should be through the rebound traits type.

[2015-05, Lenexa]

STL: You want to permit but not require rebinding?

Wakely: The current wording forces me to use the original allocator, not the rebound one.

STL: Oh, I see. Yeah, we immediately rebind.

Wakely: The edits clarify that we don't use some other allocator. The third diff is because the definitions of `EmplaceConstructible`/etc. happen with the same types. The diff to the note is because it doesn't require the value of the allocator was the one passed in.

STL: After looking at this, I think I'm comfortable with the edits. The previous Standardese was nonsense so it's pretty easy to improve upon.

Marshall: Any other opinions?

Marshall: Any objections to moving it to Ready? Review? Ready in Kona?

Wakely: My preference would be Ready. We all know this is what we're doing anyways.

Nevin: The intent won't change.

STL: I think this is the right fix.

Hwrd: I third Ready. Even if Jonathan retracts his.

Marshall: Ready!

Proposed resolution:

This wording is relative to N3485.

1. Edit 23.2.1 [container.requirements.general] paragraph 3:

For the components affected by this subclause that declare an `allocator_type`, objects stored in these components shall be constructed using the `allocator_traits<allocator_type>::rebind_traits<U>::construct` function and destroyed using the `allocator_traits<allocator_type>::rebind_traits<U>::destroy` function (20.7.8.2 [allocator.traits.members]), where `U` is either `allocator_type::value_type` or an internal type used by the

container. These functions are called only for the container's element type, not for internal types used by the container. [*Note*: This means, for example, that a node-based container might need to construct nodes containing aligned buffers and call `construct` to place the element into the buffer. — *end note*]

2. Edit 23.2.1 [container.requirements.general] paragraph 7:

[...] A copy of this allocator is used for any memory allocation **and element construction** performed, by these constructors and by all member functions, during the lifetime of each container object or until the allocator is replaced. [...]

3. Edit 23.2.1 [container.requirements.general] paragraph 13:

[...] Given **an allocator type A and given** a container type `x` having ~~an allocator type identical to A and~~ a `value_type` identical to `T` **and an allocator type identical to `allocator_traits<A>::rebind_alloc<T>`** and given an lvalue `m` of type `A`, a pointer `p` of type `T*`, an expression `v` of type (possibly `const`) `T`, and an rvalue `rv` of type `T`, the following terms are defined.

[...]

[*Note*: A container calls `allocator_traits<A>::construct(m, p, args)` to construct an element at `p` using `args`, **with `m == get_allocator()`**. The default `construct` in `std::allocator` will call `::new((void*)p) T(args)`, but specialized allocators may choose a different definition. — *end note*]

2219. *INVOKE*-ing a pointer to member with a `reference_wrapper` as the object expression

Section: 20.9.2 [func.require] **Status:** [Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2012-11-28 **Last modified:** 2015-05-06

View other [active issues](#) in [func.require].

View all other [issues](#) in [func.require].

View all issues with [Ready](#) status.

Discussion:

The standard currently requires this to be invalid:

```
#include <functional>

struct X { int i; } x;
auto f = &X::i;
auto t1 = std::ref(x);
int i = std::mem_fn(f)(t1);
```

The call expression on the last line is equivalent to `INVOKE(f, std::ref(x))` which according to 20.9.2 [func.require]p1 results in the invalid expression `(*t1).*f` because `reference_wrapper<X>` is neither an object of type `x` nor a reference to an object of type `x` nor a reference to an object of a type derived from `x`.

The same argument applies to pointers to member functions, and if they don't work with `INVOKE` it becomes harder to do all sorts of things such as:

```
call_once(o, &std::thread::join, std::ref(thr))
```

or

```
async(&std::list<int>::sort, std::ref(list));
```

The definition of `INVOKE` should be extended to handle reference wrappers.

[2013-03-15 Issues Teleconference]

Moved to Review.

The wording seems accurate, but verbose. If possible, we would like to define the kind of thing being specified so carefully as one of a number of potential language constructs in a single place. It is also possible that this clause is that single place.

[2013-04-18, Bristol]

Jonathan comments:

In the proposed resolution in the first bullet (`t1.*f`) is not valid if `t1` is a `reference_wrapper`, so we probably need a separate bullet to handle the `reference_wrapper` case.

[2014-02-14, Issaquah, Mike Spertus supplies wording]

Previous resolution from Jonathan [SUPERSEDED]:

This wording is relative to N3485.

1. Edit 20.9.2 [func.require]:

Define *INVOKE*(*f*, *t1*, *t2*, ..., *tN*) as follows:

- (*t1*.**f*)(*t2*, ..., *tN*) when *f* is a pointer to a member function of a class *T* and *t1* is an object of type *T* or a reference to an object of type *T* or a reference to an object of a type derived from *T*; *U* or an object of type `reference_wrapper<U>` or a reference to an object of type `reference_wrapper<U>` where *U* is either the type *T* or a type derived from *T*;
- ((**t1*).**f*)(*t2*, ..., *tN*) when *f* is a pointer to a member function of a class *T* and *t1* is not one of the types described in the previous item;
- *t1*.**f* when *N* == 1 and *f* is a pointer to member data of a class *T* and *t1* is an object of type *T* or a reference to an object of type *T* or a reference to an object of a type derived from *T*; *U* or an object of type `reference_wrapper<U>` or a reference to an object of type `reference_wrapper<U>` where *U* is either the type *T* or a type derived from *T*;
- ((**t1*).**f*) when *N* == 1 and *f* is a pointer to member data of a class *T* and *t1* is not one of the types described in the previous item;
- *f*(*t1*, *t2*, ..., *tN*) in all other cases.

[2014-10-01, STL adds discussion and provides an improved resolution]

Because neither *t1*.**f* nor ((**t1*).**f*) will compile when *t1* is `reference_wrapper<U>` for any *U*, we don't need to inspect *U* carefully. We can bluntly detect all `reference_wrapper`s and use `get()` for them.

We would have to be more careful if we had to deal with pointers to members of `reference_wrapper` itself. Fortunately, we don't. First, it doesn't have user-visible data members. Second, users technically can't take the addresses of its member functions (this is a consequence of 17.6.5.5 [member.functions], the Implementer's Best Friend).

While we're in the neighborhood, I recommend simplifying and clarifying the wording used to detect base/derived objects.

Previous resolution from Mike Spertus [SUPERSEDED]:

This wording is relative to N3936.

1. Edit 20.9.2 [func.require]:

Define *INVOKE*(*f*, *t1*, *t2*, ..., *tN*) as follows:

- (*t1*.**f*)(*t2*, ..., *tN*) when *f* is a pointer to a member function of a class *T* and *t1* is an object of type *T* or a reference to an object of type *T* or a reference to an object of a type derived from *T*;
- (*t1*.get()).**f*)(*t2*, ..., *tN*) when *f* is a pointer to a member function of class *T* and *t1* is an object of type `reference_wrapper<U>` where *U* is either the type *T* or a type derived from *T*.
- ((**t1*).**f*)(*t2*, ..., *tN*) when *f* is a pointer to a member function of a class *T* and *t1* is not one of the types described in the previous item;
- *t1*.**f* when *N* == 1 and *f* is a pointer to member data of a class *T* and *t1* is an object of type *T* or a reference to an object of type *T* or a reference to an object of a type derived from *T*;
- *t1*.get().**f* when *N* == 1 and *f* is a pointer to member data of a class *T* and *t1* is an object of type `reference_wrapper<U>` where *U* is either the type *T* or a type derived from *T*.
- ((**t1*).**f*) when *N* == 1 and *f* is a pointer to member data of a class *T* and *t1* is not one of the

types described in the previous item;

- $f(t_1, t_2, \dots, t_N)$ in all other cases.

[2015-02, Cologne]

Waiting for implementation experience.

[2015-05, Lenexa]

STL: latest note from Cologne, waiting for implementation experience

STL: don't think this is harder than anything else we do

MC: it does involve `mem_fn` and `invoke`

STL: my simplification was not to attempt fine-grained

STL: can ignore `pmf`

STL: can't invoke `pmf` to reference wrapper

STL: wording dated back to TR1 when there was no `decltype`

MC: should `decay_t<decltype(t1)>` be pulled out since it is in multiple places

STL: it could be handled editorially

STL: we fix function, bind, invoke

STL: have not implemented this but believe it is fine

MC: Eric F, you have worked in invoke

EF: yes, looks ok

MC: consensus move to ready

Proposed resolution:

This wording is relative to N3936.

1. Change 20.9.2 [func.require] p1 as depicted:

Define *INVOKE*($f, t_1, t_2, \dots, t_N$) as follows:

- $(t_1.*f)(t_2, \dots, t_N)$ when f is a pointer to a member function of a class T and t_1 is an object of type T or a reference to an object of type T or a reference to an object of a type derived from T is base of T , `decay_t<decltype(t1)>>::value is true`;
- $(t_1.get().*f)(t_2, \dots, t_N)$ when f is a pointer to a member function of a class T and `decay_t<decltype(t1)>` is a specialization of reference wrapper;
- $((*t_1).*f)(t_2, \dots, t_N)$ when f is a pointer to a member function of a class T and t_1 is not one of the types described in the previous item does not satisfy the previous two items;
- $t_1.*f$ when $N == 1$ and f is a pointer to member data of a class T and t_1 is an object of type T or a reference to an object of type T or a reference to an object of a type derived from T is base of T , `decay_t<decltype(t1)>>::value is true`;
- $t_1.get().*f$ when $N == 1$ and f is a pointer to member data of a class T and `decay_t<decltype(t1)>` is a specialization of reference wrapper;
- $(*t_1).*f$ when $N == 1$ and f is a pointer to member data of a class T and t_1 is not one of the types described in the previous item does not satisfy the previous two items;
- $f(t_1, t_2, \dots, t_N)$ in all other cases.

2224. Ambiguous status of access to non-live objects

Section: 17.6.4.10 [res.on.objects] **Status:** [Tentatively Ready](#) **Submitter:** Geoffrey Romer **Opened:** 2012-12-17 **Last modified:** 2015-05-05

View all other [issues](#) in [res.on.objects].

View all issues with [Tentatively Ready](#) status.

Discussion:

The standard currently does not discuss when library objects may be accessed, except in a non-normative note pertaining to synchronization in [res.on.objects], leaving it ambiguous whether single-threaded code can access a library object during its construction or destruction. For example, there is a reasonable question as to what happens if the deleter supplied to a `unique_ptr` transitively accesses the `unique_ptr` itself during `unique_ptr`'s destruction; a straightforward reading suggests that this is permitted, and that the deleter will see the `unique_ptr` still holding the originally stored pointer, but consensus on the LWG reflector indicates this was not the intent (see discussion beginning with [c++std-lib-33362](#)).

[2013-03-15 Issues Teleconference]

Moved to Open.

Geoffrey will provide an example that clearly highlights the issue.

[2013-03-19 Geoffrey provides revised resolution and an example]

I contend that the most straightforward reading of the current standard requires the following example code to print "good" (because `~unique_ptr` is not specified to modify the state of the internal pointer), but the consensus on the reflector was that its behavior should be undefined.

This example also shows that, contrary to a comment in the telecon, the PR is not tautological. 12.7 [class.ctor]/p4 explicitly permits member function calls during destruction, so the behavior of this code is well-defined as far as the core language is concerned, despite the fact that it accesses a library object after the end of the object's lifetime. If we want this code to have undefined behavior, we need to specify that at the library level.

```
#include <memory>
#include <iostream>

class A;

struct B {
    std::unique_ptr<A> a;
};

struct A {
    B* b;
    ~A() {
        if (b->a.get() == this) {
            std::cout << "good" << std::endl;
        }
    }
};

int main() {
    B b;
    b.a.reset(new A);
    b.a->b = &b;
}
```

Previous resolution:

1. Change the title of sub-clause 17.6.4.10 [res.on.objects] as indicated:

~~Shared objects and the library~~ **Library object access** [res.on.objects]

2. Edit 17.6.4.10 [res.on.objects] p2 as indicated:

-2- ~~[Note: In particular, the program is required to ensure that completion of the constructor of any object of a class type defined in the standard library happens before any other member function invocation on that object and, unless otherwise specified, to ensure that completion of any member function invocation other than destruction on such an object happens before destruction of that object. This applies even to objects such as mutexes intended for thread synchronization. — end note]~~ **If an object of a standard library type is accessed outside of the object's lifetime (3.8 [basic.lifel]), the behavior is undefined unless otherwise specified.**

[2014 Urbana]

STL: is this resolved by our change to the reentrancy rules? [LWG [2414](#)]

GR: don't think that solves the multi-threaded case

MC: I like changing the note to normative text

GR: uses the magic "happens before" words, and "access" is magic too

JW: I like this. strict improvement, uses the right wording we have to say this properly

STL: I like the last sentence of the note, could we add that as a new note at the end?

So add "[Note: This applies even to objects such as mutexes intended for thread synchronization.]" to the end and move to Ready

Proposed resolution:

This wording is relative to N3485.

1. Change the title of sub-clause 17.6.4.10 [res.on.objects] as indicated:

~~Shared objects and the library~~ **Library object access** [res.on.objects]

2. Edit 17.6.4.10 [res.on.objects] p2 as indicated: *[Editorial remark: The motivation, is to be more precise about the meaning of "outside the object's lifetime" in the presence of threads — end editorial remark]*

~~-2- [Note: In particular, the program is required to ensure that completion of the constructor of any object of a class type defined in the standard library happens before any other member function invocation on that object and, unless otherwise specified, to ensure that completion of any member function invocation other than destruction on such an object happens before destruction of that object. This applies even to objects such as mutexes intended for thread synchronization. — end note]~~
If an object of a standard library type is accessed, and the beginning of the object's lifetime (3.8 [basic.life]) does not happen before the access, or the access does not happen before the end of the object's lifetime, the behavior is undefined unless otherwise specified. [Note: This applies even to objects such as mutexes intended for thread synchronization. — end note]

2234. `assert()` should allow usage in constant expressions

Section: 19.3 [assertions] **Status:** [Tentatively Ready](#) **Submitter:** Daniel Krügler **Opened:** 2013-01-12 **Last modified:** 2015-09-27

View other [active issues](#) in [assertions].

View all other [issues](#) in [assertions].

View all issues with [Tentatively Ready](#) status.

Discussion:

It is unclear from the current specification whether `assert()` expressions can be used in (potential) constant expressions. As an example consider the implementation of a `constexpr` function:

```
#include <cassert>

template<class T, unsigned N>
struct array {
    T data[N];
    constexpr const T& operator[](unsigned i) const {
        return assert(i < N), data[i];
    }
};

int main() {
    constexpr array<int, 3> ai = {1, 2, 3};
    constexpr int i = ai[0];
    int j = ai[0];
    // constexpr int k = ai[5];
}
```

The first question is whether this program is guaranteed well-formed? A second question is whether it would be guaranteed to be ill-formed, if we uncomment the last code line in `main()`?

The wording in 19.3 [assertions] doesn't add anything significant to the C99 wording. From the C99 specification (7.2 p1 and 7.2.1.1 p2) we get already some valuable guarantees:

- The expression `assert(e)` is a void expression for all expressions `e` independent of the definition of `NDEBUG`.
- If `NDEBUG` is defined, `assert(e)` is equivalent to the expression `void()` (or anything that cannot be distinguished from that).

The current wording does not yet *guarantee* that `assert` expressions can be used in constant expressions, but all tested implementations (gcc, MSVC) would already support this use-case. It seems to me that this should be possible without giving `assert` a special meaning for the core language.

As a related comment it should be added, that there is a core language [proposal](#) that intends to relax some current constraints for `constexpr` functions and `literal` types. The most interesting one (making `void` a literal types and allowing for expression-statements) would simplify the motivating example implementation of `operator[]` to:

```
constexpr const T& operator[](unsigned i) const {  
    assert(i < N);  
    return data[i];  
};
```

[2013-03-15 Issues Teleconference]

Moved to Open.

We are still gaining experience with `constexpr` as a language feature, and there may be work in Evolution that would help address some of these concerns. Defer discussion until we have a group familiar with any evolutionary direction.

[2014-06-08, Daniel comments and suggests wording]

After approval of [N3652](#), `void` is now a literal type and `constexpr` functions can contain multiple statements, so this makes the guarantee that `assert` expressions are per-se `constexpr`-friendly even more relevant. A possible wording form could be along the lines of:

For every core constant expression e of scalar type that evaluates to `true` after being contextually converted to `bool`, the expression `assert(e)` shall be a prvalue core constant expression of type `void`.

Richard Smith pointed out some weaknesses of this wording form, for example it would not guarantee to require the following example to work:

```
constexpr void check(bool b) { assert(b); }
```

because `b` is not a core constant expression in this context.

He suggested improvements that lead to the wording form presented below (any defects mine).

[Lenexa 2015-05-05]

MC : ran into this

Z : Is it guaranteed to be an expression?

MC : clarifies that `assert` runs at runtime, not sure what it does at compile time

STL : c standard guarantees its an expression and not a whole statement, so comma chaining it is ok

HH : Some implementations work as author wants it to

STL : also doing this as `constexpr`

DK/STL : discussing how this can actually work

HH : GCC 5 also implements it. We have implementor convergence

MC : Wants to do this without giving `assert` a special meaning

STL : NDEBUG being defined where `assert` appears is not how `assert` works. This is bug in wording. Should be "when `assert` is defined" or something like that. ... is a constant subexpression if `NDEBUG` is defined at the point where `assert` is last defined or redefined."

Would like to strike the "either" because ok if both debug or assertion is true. We want inclusive-or here

MC : is redefined needed?

STL : my mental model is its defined once and then redefined

HH : wants to up to P2

Z/STL : discussing how wording takes care of how/when `assert` is defined/redefined

STL/WB : discussing whether to move to ready or review. -> Want to move it to ready.

ask for updated wording

p3 -> p2

plan to go to ready after checking wording

[Telecom 2015-06-30]

HH: standardizing existing practice

MC: what about the comment from Lenexa about striking "either"?

HH: all three implementations accept it

MC: update issue to strike "either" and move to Tentatively Ready

Proposed resolution:

This wording is relative to N3936.

Previous resolution [SUPERSEDED]:

1. Introduce the following new definition to the existing list in 17.3 [definitions]: *[Drafting note: If LWG 2296 is accepted before this issue, the accepted wording for the new definition should be used instead — end drafting note]*

constant subexpression [defns.const.subexpr]

an expression whose evaluation as subexpression of a *conditional-expression* *CE* (5.16 [expr.cond]) would not prevent *CE* from being a core constant expression (5.20 [expr.const]).

2. Insert a new paragraph following 19.3 [assertions] p1 as indicated:

-? An expression `assert(E)` is a constant subexpression ([defns.const.subexpr]), if either

- `NDEBUG` is defined at the point where `assert(E)` appears, or
- `E` contextually converted to `bool` (4 [conv]), is a constant subexpression that evaluates to the value `true`.

1. Introduce the following new definition to the existing list in 17.3 [definitions]: *[Drafting note: If LWG 2296 is accepted before this issue, the accepted wording for the new definition should be used instead — end drafting note]*

constant subexpression [defns.const.subexpr]

an expression whose evaluation as subexpression of a *conditional-expression* *CE* (5.16 [expr.cond]) would not prevent *CE* from being a core constant expression (5.20 [expr.const]).

2. Insert a new paragraph following 19.3 [assertions] p1 as indicated:

-? An expression `assert(E)` is a constant subexpression ([defns.const.subexpr]), if

- `NDEBUG` is defined at the point where `assert(E)` appears, or
- `E` contextually converted to `bool` (4 [conv]), is a constant subexpression that evaluates to the value `true`.

2244. Issue on `basic_istream::seekg`

Section: 27.7.2.3 [istream.unformatted] **Status:** [Ready](#) **Submitter:** Juan Soulie **Opened:** 2013-03-04 **Last modified:** 2015-05-22

View other [active issues](#) in [istream.unformatted].

View all other [issues](#) in [istream.unformatted].

View all issues with [Ready](#) status.

Discussion:

When issue 1445 was resolved by adopting N3168, it exposed the need to modify both overloads of `basic_istream::seekg` (by inserting "the function clears eofbit," after "except that"), but the fix applied to the text apparently forgets the second overload at 27.7.2.3 [istream.unformatted] p43.

[2013-10-17: Daniel provides concrete wording]

It seems that the tiny sentence "SIMILARLY for 27.7.1.3/43 (`seekg`)." had been overlooked. I agree that the wording needs to be applied here as well.

[2015-05-06 Lenexa: Move to Ready]

MC: This was just missed when we added "the function first clears eofbit" to the other overload, Daniel agrees. Editing mistake.

Move to Ready, consensus.

Proposed resolution:

This wording is relative to N3691.

1. Change 27.7.2.3 [istream.unformatted] p43 as indicated:

```
basic_istream<charT,traits>& seekg(off_type off, ios_base::seekdir dir);
```

-43- *Effects:* Behaves as an unformatted input function (as described in 27.7.2.3, paragraph 1), except that **the function first clears eofbit**, it does not count the number of characters extracted, and does not affect the value returned by subsequent calls to `gcount()`. [...]

[2250](#). Follow-up On Library Issue 2207

Section: 20.6.1 [bitset.cons], 20.6.2 [bitset.members], 21.4.2 [string.cons], 21.4.6 [string.modifiers], 21.4.7 [string.ops] **Status:** [Ready](#) **Submitter:** Frank Birbacher **Opened:** 2013-04-18 **Last modified:** 2015-09-25

View all other [issues](#) in [bitset.cons].

View all issues with [Ready](#) status.

Discussion:

Similar to LWG [2207](#) there are several other places where the "Requires" clause precludes the "Throws" condition. Searching for the `out_of_range` exception to be thrown, the following have been found (based on the working draft N3485):

1. 20.6.1 [bitset.cons] p3+4
2. 20.6.2 [bitset.members] p13+14 (set)
3. 20.6.2 [bitset.members] p19+20 (reset)
4. 20.6.2 [bitset.members] p27+28 (flip)
5. 20.6.2 [bitset.members] p41+42 (test)
6. 21.4.2 [string.cons] p3+4
7. 21.4.6.2 [string::append] p3+4
8. 21.4.6.3 [string::assign] p4+5
9. 21.4.6.4 [string::insert] p1+2, p5+6, p9+10 (partially)
10. 21.4.6.5 [string::erase] p1+2
11. 21.4.6.6 [string::replace] p1+2, p5+6, p9+10 (partially)
12. 21.4.6.7 [string::copy] p1+2
13. 21.4.7.8 [string::substr] p1+2

[2013-10-15: Daniel provides wording]

In addition to the examples mentioned in the discussion, a similar defect exists for `thread's join()` and `detach` functions (see 30.3.1.5 [thread.thread.member]). The suggested wording applies a similar fix for these as well.

[2015-05, Lenexa]

STL : likes it

DK : does it change behavior?

Multiple : no

Move to ready? Unanimous

Proposed resolution:

This wording is relative to N3936.

1. Modify 20.6.1 [bitset.cons] as indicated: *[Editorial comment: The wording form used to ammend the Throws element is borrowed from a similar style used in 21.4.6.6 [string::replace] p10]*

```
template <class charT, class traits, class Allocator>
explicit
bitset(const basic_string<charT, traits, Allocator>& str,
        typename basic_string<charT, traits, Allocator>::size_type pos = 0,
        typename basic_string<charT, traits, Allocator>::size_type n =
```



```
basic_string<charT, traits, Allocator>::npos,
charT zero = charT('0'), charT one = charT('1'));
```

~~-3- Requires: pos <= str.size();~~

~~-4- Throws: out_of_range if pos > str.size()~~ **or invalid_argument if an invalid character is found (see below).**

~~-5- Effects: Determines the effective length rlen of the initializing string as the smaller of n and str.size() - pos.~~

The function then throws invalid_argument if any of the rlen characters in str beginning at position pos is other than zero or one. The function uses traits::eq() to compare the character values.

[...]

2. Modify 20.6.2 [bitset.members] as indicated:

```
bitset<N>& set(size_t pos, bool val = true);
```

~~-13- Requires: pos is valid~~

~~-14- Throws: out_of_range if pos does not correspond to a valid bit position.~~

[...]

```
bitset<N>& reset(size_t pos);
```

~~-19- Requires: pos is valid~~

~~-20- Throws: out_of_range if pos does not correspond to a valid bit position.~~

[...]

```
bitset<N>& flip(size_t pos);
```

~~-27- Requires: pos is valid~~

~~-28- Throws: out_of_range if pos does not correspond to a valid bit position.~~

[...]

```
bool test(size_t pos) const;
```

~~-41- Requires: pos is valid~~

~~-42- Throws: out_of_range if pos does not correspond to a valid bit position.~~

[...]

3. Modify 21.4.2 [string.cons] as indicated:

```
basic_string(const basic_string& str,
             size_type pos, size_type n = npos,
             const Allocator& a = Allocator());
```

~~-3- Requires: pos <= str.size()~~

~~-4- Throws: out_of_range if pos > str.size().~~

4. Modify 21.4.4 [string.capacity] as indicated:

```
void resize(size_type n, charT c);
```

~~-6- Requires: n <= max_size()~~

~~-7- Throws: length_error if n > max_size().~~

5. Modify 21.4.6.2 [string::append] as indicated:

```
basic_string&
append(const basic_string& str, size_type pos, size_type n = npos);
```

~~-3- Requires: pos <= str.size()~~

~~-4- Throws: out_of_range if pos > str.size().~~

6. Modify 21.4.6.3 [string::assign] as indicated:

```
basic_string&
assign(const basic_string& str, size_type pos,
       size_type n = npos);
```

~~-5- Requires: pos <= str.size()~~

~~-6- Throws: out_of_range if pos > str.size().~~

7. Modify 21.4.6.4 [string::insert] as indicated: *[Editorial note: The first change suggestion is also a bug fix of the current wording, because (a) the function has parameter pos1 but the semantics refers to pos and (b) it is possible that this function can throw length_error, see p10]*

```
basic_string&
insert(size_type pos1, const basic_string& str);
```

~~-1- Requires: pos <= size();~~

~~-2- Throws: out_of_range if pos > size();~~

~~-3- Effects: Calls~~Equivalent to: `return insert(pos, str.data(), str.size());`~~;~~

~~-4- Returns: *this;~~

```
basic_string&
insert(size_type pos1, const basic_string& str,
       size_type pos2, size_type n = npos);
```

~~-5- Requires: pos1 <= size() and pos2 <= str.size();~~

~~-6- Throws: out_of_range if pos1 > size() OR pos2 > str.size().~~

[...]

```
basic_string&
insert(size_type pos, const charT* s, size_type n);
```

~~-9- Requires: s points to an array of at least n elements of charT and pos <= size().~~

~~-10- Throws: out_of_range if pos > size() OR length_error if size() + n > max_size().~~

[...]

```
basic_string&
insert(size_type pos, const charT* s);
```

~~-13- Requires: pos <= size() and~~ s points to an array of at least traits::length(s) + 1 elements of charT.

~~-14- Effects: Equivalent to~~ `return insert(pos, s, traits::length(s));`~~;~~

~~-15- Returns: *this;~~

8. Modify 21.4.6.5 [string::erase] as indicated:

```
basic_string& erase(size_type pos = 0, size_type n = npos);
```

~~-1- Requires: pos <= size()~~

~~-2- Throws: out_of_range if pos > size().~~

[...]

9. Modify 21.4.6.6 [string::replace] as indicated: *[Editorial note: The first change suggestion is also a bug fix of the current wording, because it is possible that this function can throw length_error, see p10]*

```
basic_string&
replace(size_type pos1, size_type n1,
       const basic_string& str);
```

~~-1- Requires: pos1 <= size();~~

~~-2- Throws: out_of_range if pos1 > size().~~

~~-3- Effects: Calls Equivalent to return replace(pos1, n1, str.data(), str.size()).~~

~~-4- Returns: *this.~~

```
basic_string&
replace(size_type pos1, size_type n1,
        const basic_string& str,
        size_type pos2, size_type n = npos);
```

~~-5- Requires: pos1 <= size() and pos2 <= str.size().~~

~~-6- Throws: out_of_range if pos1 > size() OR pos2 > str.size().~~

[...]

```
basic_string&
replace(size_type pos1, size_type n1, const charT* s, size_type n2);
```

~~-9- Requires: pos1 <= size() and s points to an array of at least n2 elements of charT.~~

~~-10- Throws: out_of_range if pos1 > size() OR length_error if the length of the resulting string would exceed max_size() (see below).~~

[...]

```
basic_string&
replace(size_type pos, size_type n, const charT* s);
```

~~-13- Requires: pos <= size() and s points to an array of at least traits::length(s) + 1 elements of charT.~~

~~-14- Effects: Equivalent to return replace(pos, n, s, traits::length(s)).~~

~~-15- Returns: *this.~~

10. Modify 21.4.6.7 [string::copy] as indicated:

```
size_type copy(charT* s, size_type n, size_type pos = 0) const;
```

~~-1- Requires: pos <= size()~~

~~-2- Throws: out_of_range if pos > size().~~

[...]

11. Modify 21.4.7.8 [string::substr] as indicated:

```
basic_string substr(size_type pos = 0, size_type n = npos) const;
```

~~-1- Requires: pos <= size()~~

~~-2- Throws: out_of_range if pos > size().~~

[...]

12. Modify 30.3.1.5 [thread.thread.member] as indicated:

```
void join();
```

~~-3- Requires: joinable() is true.~~

[...]

~~-7- Throws: system_error when an exception is required (30.2.2).~~

~~-8- Error conditions:~~

- [...]
- invalid_argument — if the thread is not joinable.

```
void detach();
```

~~-9- Requires: joinable() is true.~~

[...]

-12- Throws: `system_error` when an exception is required (30.2.2).

-13- Error conditions:

- [...]
- `invalid_argument` — if the thread is not joinable.

2259. Issues in 17.6.5.5 rules for member functions

Section: 17.6.5.5 [member.functions] **Status:** [Ready](#) **Submitter:** Richard Smith **Opened:** 2013-05-12 **Last modified:** 2015-09-25

View all other [issues](#) in [member.functions].

View all issues with [Ready](#) status.

Discussion:

17.6.5.5 [member.functions] p2 says:

"An implementation may declare additional non-virtual member function signatures within a class:

- by adding arguments with default values to a member function signature; [Footnote: Hence, the address of a member function of a class in the C++ standard library has an unspecified type.] [*Note: An implementation may not add arguments with default values to virtual, global, or non-member functions. — end note*]
- by replacing a member function signature with default values by two or more member function signatures with equivalent behavior; and
- by adding a member function signature for a member function name."

1. This wording is not using the correct terminology. "by adding arguments with default values" presumably means "by adding parameters with default arguments", and likewise throughout.
2. This paragraph only allows an implementation to declare "additional" signatures, but the first bullet is talking about replacing a standard signature with one with additional parameters.
3. None of these bullets allows a member function with no *ref*-qualifier to be replaced by signatures with *ref*-qualifiers (a situation which was just discussed on std-proposals), and likewise for *cv*-qualifiers. Presumably that is not intentional, and such changes should be permissible.

I think the first two items are probably editorial, since the intent is clear.

[2013-12-11 Richard provides concrete wording]

[2015-05, Lenexa]

JW: I don't like that this loses the footnote about the address of member functions having an unspecified type, the footnote is good to be able to point to as an explicit clarification of one consequence of the normative wording.

MC: so we want to keep the footnote

STL: doesn't need to be a footnote, can be an inline Note

JW: does this have any impact on our ability to add totally different functions with unrelated names, not described in the standard?

MC: no, the old wording didn't refer to such functions anyway

Move to Ready and include in motion on Friday?

9 in favor, 0 opposed, 2 abstention

Proposed resolution:

This wording is relative to N3797.

1. Merge 17.6.5.5 [member.functions]p2+3 as indicated:

~~-2- An implementation may declare additional non-virtual member function signatures within a class:~~

- ~~by adding arguments with default values to a member function signature;188 [*Note: An implementation*~~

may not add arguments with default values to virtual, global, or non-member functions. — *end note*]

- by replacing a member function signature with default values by two or more member function signatures with equivalent behavior; and
- by adding a member function signature for a member function name.

3- A call to a member function signature described in the C++ standard library behaves as if the implementation declares no additional member function signatures. [Footnote: A valid C++ program always calls the expected library member function, or one with equivalent behavior. An implementation may also define additional member functions that would otherwise not be called by a valid C++ program.] For a non-virtual member function described in the C++ standard library, an implementation may declare a different set of member function signatures, provided that any call to the member function that would select an overload from the set of declarations described in this standard behaves as if that overload were selected. [Note: For instance, an implementation may add parameters with default values, or replace a member function with default arguments with two or more member functions with equivalent behavior, or add additional signatures for a member function name. — *end note*]

2273. `regex_match` ambiguity

Section: 28.11.2 [re.alg.match] **Status:** [Tentatively Ready](#) **Submitter:** Howard Hinnant **Opened:** 2013-07-14 **Last modified:** 2015-09-14

View all other [issues](#) in [re.alg.match].

View all issues with [Tentatively Ready](#) status.

Discussion:

28.11.2 [re.alg.match] p2 in describing `regex_match` says:

-2- *Effects:* Determines whether there is a match between the regular expression `e`, and all of the character sequence `[first,last)`. The parameter `flags` is used to control how the expression is matched against the character sequence. Returns true if such a match exists, false otherwise.

It has come to my attention that different people are interpreting the first sentence of p2 in different ways:

1. If a search of the input string using the regular expression `e` matches the entire input string, `regex_match` should return true.
2. Search the input string using the regular expression `e`. Reject all matches that do not match the entire input string. If a such a match is found, return true.

The difference between these two subtly different interpretations is found using the following ECMAScript example:

```
std::regex re("Get|GetValue");
```

Using `regex_search`, this `re` can never match the input string "GetValue", because ECMA specifies that alternations are ordered, not greedy. As soon as "Get" is matched in the left alternation, the matching algorithm stops.

Using definition 1, `regex_match` would return false for an input string of "GetValue".

However definition 2 alters the grammar and appears equivalent to augmenting the regex with a trailing '\$', which is an anchor that specifies, reject any matches which do not come at the end of the input sequence. So, using definition 2, `regex_match` would return true for an input string of "GetValue".

My opinion is that it would be strange to have `regex_match` return true for a string/regex pair that `regex_search` could never find. I.e. I favor definition 1.

John Maddock writes:

The intention was always that `regex_match` would reject any match candidate which didn't match the entire input string. So it would find `GetValue` in this case because the "Get" alternative had already been rejected as not matching. Note that the comparison with ECMA script is somewhat moot, as ECMAScript defines the regex grammar (the bit we've imported), it does not define anything like `regex_match`, nor do we import from ECMAScript the behaviour of that function. So IMO the function should behave consistently regardless of the regex dialect chosen. Saying "use awk regexes" doesn't cut it, because that changes the grammar in other ways.

(John favors definition 2).

We need to clarify 28.11.2 [re.alg.match]/p2 in one of these two directions.

[2014-06-21, Rapperswil]

AM: I think there's a clear direction and consensus we agree with John Maddock's position, and if noone else thinks we need the other function I won't ask for it.

Marshall Clow and STL to draft.

[2015-06-10, Marshall suggests concrete wording]

[2015-01-11, Telecom]

Move to Tentatively Ready

Proposed resolution:

This wording is relative to N4527.

1. Change 28.11.2 [re.alg.match]/2, as follows:

```
template <class BidirectionalIterator, class Allocator, class charT, class traits>
bool regex_match(BidirectionalIterator first, BidirectionalIterator last,
                 match_results<BidirectionalIterator, Allocator>& m,
                 const basic_regex<charT, traits>& e,
                 regex_constants::match_flag_type flags =
                 regex_constants::match_default);
```

-1- *Requires*: The type BidirectionalIterator shall satisfy the requirements of a Bidirectional Iterator (24.2.6).

-2- *Effects*: Determines whether there is a match between the regular expression *e*, and all of the character sequence [first,last). The parameter *flags* is used to control how the expression is matched against the character sequence. When determining if there is a match, only potential matches that match the entire character sequence are considered. Returns true if such a match exists, false otherwise. Example:

```
std::regex re("Get|GetValue");
std::cmatch m;
regex_search("GetValue", m, re); // returns true, and m[0] contains "Get"
regex_match("GetValue", m, re); // returns true, and m[0] contains "GetValue"
regex_search("GetValues", m, re); // returns true, and m[0] contains "Get"
regex_match("GetValues", m, re); // returns false
```

— end example

[...]

2336. is_trivially_constructible/is_trivially_assignable traits are always false

Section: 20.10.4.3 [meta.unary.prop] **Status:** [Ready](#) **Submitter:** Daniel Krüglér **Opened:** 2013-10-01 **Last modified:** 2015-05-08

View other [active issues](#) in [meta.unary.prop].

View all other [issues](#) in [meta.unary.prop].

View all issues with [Ready](#) status.

Discussion:

In 20.10.4.3 [meta.unary.prop] we have traits to allow testing for triviality of specific operations, such as `is_trivially_constructible` and `is_trivially_assignable` (and their derived forms), which are specified in terms of the following initialization and assignment, respectively:

```
T t(create<Args>()...);

declval<T>() = declval<U>()
```

The wording that describes the way how triviality is deduced, is in both cases of the same form:

[...] and the variable definition/assignment, as defined by `is_constructible/is_assignable`, is known to call no operation that is not trivial (3.9, 12).

The problematic part of this wording is that both definitions are specified in terms of an "object construction" function `create` or `declval`, respectively, (The former being a conceptual function, the latter being a library function), but for none of these functions we can assume that they could be considered as trivial — only special member functions can have this property and none of these is one. This problem became obvious, when the similar issue LWG [2298](#) in regard to `is_nothrow_constructible` was opened.

A possible approach to solve this specification problem is to make a blanket statement for sub-clause 20.10.4.3 [meta.unary.prop] that these helper functions are considered trivial for the purpose of defining these traits.

Using this kind of wording technique can also be used to get rid of the additional helper function template `create`, which is currently needed for the `is_convertible` and the `is_constructible` traits, because both traits are specified in terms of contexts where technically the corresponding "object construction" function would be considered as odr-used. This is problematic, because these traits are defined in terms of well-formed code and odr-using `declval` would make the program ill-formed (see 20.2.5 [declval]). So extending above blanket statement to consider `std::declval<T>()` as not odr-used in the context of the corresponding trait definition would allow for replacing `create` by `declval`.

[2015-05, Lenexa]

STL: would you consider moving the change to 20.10 as editorial or are you uncomfortable with it?

JW: this sounds a viable editorial change

VV: I guarantee you that moving it doesn't change anything

MC: how about this: we move it to Ready as is and if we conclude moving it is editorial we can do it and if not open an issue

STL: I would like to guarantee that the lifting happens

JW: I do that! If it goes in I move it up

MC: move to Ready: in favor: 15, opposed: 0, abstain: 1

Proposed resolution:

This wording is relative to N3936.

1. Add a new paragraph after 20.10.4.3 [meta.unary.prop] p3 as indicated: *[Editorial note: The first change in 20.10.4.3 [meta.unary.prop] p3 is recommended, because technically a Clause is always a "main chapter" — such as Clause 20 — but every child of a Clause or sub-clause is a sub-clause]*

[...]

-3- For all of the class templates `x` declared in this **Clause****sub-clause**, instantiating that template with a template-argument that is a class template specialization may result in the implicit instantiation of the template argument if and only if the semantics of `x` require that the argument must be a complete type.

-?- For the purpose of defining the templates in this sub-clause, a function call expression `declval<T>()` for any type `T` is considered to be a trivial (3.9 [basic.types], 12 [special]) function call that is not an odr-use (3.2 [basic.def.odr]) of `declval` in the context of the corresponding definition notwithstanding the restrictions of 20.2.5 [declval].

[...]

2. Modify 20.10.4.3 [meta.unary.prop] p7 as indicated:

-7- **Given the following function prototype:**

```
template <class T>
typename add_rvalue_reference<T>::type create();
```

†The predicate condition for a template specialization `is_constructible<T, Args...>` shall be satisfied if and only if the following variable definition would be well-formed for some invented variable `t`:

```
T t(create declval<Args>()...);
```

[...]

3. Add a new paragraph after 20.10.6 [meta.rel] p2 as indicated: *[Editorial note: Technically we don't need the guarantee of "a trivial function call" for the type relationship predicates at the very moment, but it seems more robust and consistent to have the exact same guarantee here as well]*

[...]

-2- [...]

-2- For the purpose of defining the templates in this sub-clause, a function call expression `declval<T>()` for any type `T` is considered to be a trivial (3.9 [basic.types], 12 [special]) function call that is not an odr-use (3.2 [basic.def.odr]) of `declval` in the context of the corresponding definition notwithstanding the restrictions of 20.2.5 [declval].

[...]

4. Modify 20.10.6 [meta.rel] p4 as indicated:

-4- Given the following function prototype:

```
template <class T>
typename add_rvalue_reference<T>::type create();
```

The predicate condition for a template specialization `is_convertible<From, To>` shall be satisfied if and only if the return expression in the following code would be well-formed, including any implicit conversions to the return type of the function:

```
To test() {
    return create<declval<From>>();
}
```

[...]

2353. `std::next` is over-constrained

Section: 24.4.4 [iterator.operations] **Status:** [Ready](#) **Submitter:** Eric Niebler **Opened:** 2013-12-24 **Last modified:** 2015-05-22

View all other [issues](#) in [iterator.operations].

View all issues with [Ready](#) status.

Discussion:

In LWG [1011](#), `std::next` and `std::prev` were changed from accepting `InputIterator` to accepting `ForwardIterator`. This needlessly excludes perfectly legitimate use cases. Consider the following hypothetical range-based implementation of `drop`, which creates a view of a range without the first `n` elements:

```
template<typename Distance, typename InputRange>
iterator_range<range_iterator_t<InputRange>>
drop(Distance n, InputRange& rng)
{
    return make_iterator_range(
        std::next(std::begin(rng), n),
        std::end(rng)
    );
}
```

I believe this to be a legitimate use case that is currently outlawed by the standard without cause. See the discussion beginning at [c++std-lib-35313](#) for an in-depth discussion of the issue, in which Howard Hinnant agreed that it was a defect.

(Some discussion then ensued about whether an overload should be added that only accepts rvalue `InputIterators` to avoid the surprise that issue [1011](#) sought to address. I make no such attempt, nor do I believe it to be necessary.)

Suggested resolution:

Back out the resolution of [1011](#).

[Lenexa 2015-05-07: Move to Ready]

Proposed resolution:

This wording is relative to N3797.

1. Change 24.3 [iterator.synopsis], header `<iterator>` synopsis, and 24.4.4 [iterator.operations] before p.6 as indicated:

```
template <class ForwardInputIterator>
```

```
ForwardInputIterator next(ForwardInputIterator x,
    typename std::iterator_traits<ForwardInputIterator>::difference_type n = 1);
```

2367. pair and tuple are not correctly implemented for is_constructible with no args

Section: 20.10.4.3 [meta.unary.prop] **Status:** [Ready](#) **Submitter:** Howard Hinnant **Opened:** 2014-02-19 **Last modified:** 2015-05-08

View other [active issues](#) in [meta.unary.prop].

View all other [issues](#) in [meta.unary.prop].

View all issues with [Ready](#) status.

Discussion:

Consider:

```
struct X
{
    X() = delete;
};

int main()
{
    typedef std::pair<int, X> P;
    static_assert(!std::is_constructible<P>::value, "");
    static_assert(!std::is_default_constructible<P>::value, "");
    typedef std::tuple<int, X> T;
    static_assert(!std::is_constructible<T>::value, "");
    static_assert(!std::is_default_constructible<T>::value, "");
}
```

For me these static_asserts fail. And worse than that, even asking the question fails (as opposed to gets the wrong answer):

```
assert(!std::is_constructible<P>::value);
```

In file included from test.cpp:2:

```
error:
    call to deleted constructor of 'X'
    pair() : first(), second() {}
              ^
note: function has been explicitly marked deleted here
    X() = delete;
    ^
1 error generated.
```

This can be solved by specializing is_constructible on pair and tuple for zero Args:

```
template <class T, class U>
struct is_constructible<pair<T, U>>
    : integral_constant<bool, is_default_constructible<T>::value &&
                             is_default_constructible<U>::value>
{};

template <class ...T>
struct is_constructible<tuple<T...>>
    : integral_constant<bool,
        _all<is_default_constructible<T>::value...>::value>
{};
```

Now everything just works.

[2014-05-14, Daniel comments]

The proposed resolution is incomplete, because it wouldn't work for cv-qualified objects of pair or for references of them during reference-initialization.

I would like to point out that the approach suggested in [N3739](#) can be easily extended to solve the problem without need to muddle with specializing is_constructible:

```
template<class U1 = T1, class U2 = T2,
    typename enable_if<
        is_default_constructible<U1>::value && is_default_constructible<U2>::value
    , bool>::type = false
```

```
>
constexpr pair();
```

The new wording proposal represents an alternative wording change that I would strongly prefer.

Previous resolution from Howard [SUPERSEDED]:

This wording is relative to N3936.

1. Add to 20.3.3 [pairs.spec]:

```
template <class T, class U>
struct is_constructible<pair<T, U>>
: integral_constant<bool, is_default_constructible<T>::value &&
                        is_default_constructible<U>::value>
{};
```

2. Add to 20.4.2.9 [tuple.special]:

```
template <class ...T>
struct is_constructible<tuple<T...>>
: integral_constant<bool, see below>
{};
```

*?- The second argument to integral_constant shall be true if for each T_i ,
is_default_constructible< T_i >::value is true.*

[2015-05, Lenexa]

STL: I object to this resolution due to British spelling of behavior

JW: we already have other places of this spelling

VV: the easy resolution is to remove the notes

MC: if that's all we want to change: put it in and make the editorial change of removing the note

VV: the other paper doesn't make any of these changes so it would be consistent

JW: this make me want even more the features of having constructors doing the Right Thing - I haven't written up the request to do something like that

VV: so it would be an aggregate reflecting the properties of the constituting types

JW: I should write that up

MC: any objection to move to ready? in favor: 16, opposed: 0, abstain: 1

Proposed resolution:

This wording is relative to N3936.

1. Change 20.3.2 [pairs.pair] around p3 as indicated:

```
constexpr pair();
```

~~-3- *Requires:* is_default_constructible<first_type>::value is true and
is_default_constructible<second_type>::value is true.~~

~~-4- *Effects:* Value-initializes first and second.~~

*?- *Remarks:* This constructor shall not participate in overload resolution unless
is_default_constructible<first_type>::value is true and is_default_constructible<second_type>::value
is true. [Note: This behaviour can be implemented by a constructor template with default template
arguments — end note].*

2. Change 20.4.2.1 [tuple.cnstr] around p4 as indicated:

```
constexpr tuple();
```

~~-4- *Requires:* is_default_constructible< T_i >::value is true for all i .~~

~~-5- *Effects:* Value initializes each element.~~

*?- *Remarks:* This constructor shall not participate in overload resolution unless
is_default_constructible< T_i >::value is true for all i . [Note: This behaviour can be implemented by a
constructor template with default template arguments — end note].*

[2380](#). May `<cstdlib>` provide `long ::abs(long)` and `long long ::abs(long long)`?

Section: 17.6.1.2 [headers] **Status:** [Ready](#) **Submitter:** Richard Smith **Opened:** 2014-03-31 **Last modified:** 2015-05-05

View all other [issues](#) in [headers].

View all issues with [Ready](#) status.

Discussion:

D.5 [depr.c.headers] p3 says:

[Example: The header `<cstdlib>` assuredly provides its declarations and definitions within the namespace `std`. It may also provide these names within the global namespace. The header `<stdlib.h>` assuredly provides the same declarations and definitions within the global namespace, much as in the C Standard. It may also provide these names within the namespace `std`. — end example]

This suggests that `<cstdlib>` may provide `::abs(long)` and `::abs(long long)`. But this seems like it might contradict the normative wording of 17.6.1.2 [headers] p4:

Except as noted in Clauses 18 through 30 and Annex D, the contents of each header *cname* shall be the same as that of the corresponding header *name.h*, as specified in the C standard library (1.2) or the C Unicode TR, as appropriate, as if by inclusion. In the C++ standard library, however, the declarations (except for names which are defined as macros in C) are within namespace scope (3.3.6) of the namespace `std`. It is unspecified whether these names are first declared within the global namespace scope and are then injected into namespace `std` by explicit using-declarations (7.3.3).

Note that this allows `<cstdlib>` to provide `::abs(int)`, but does not obviously allow `::abs(long)` nor `::abs(long long)`, since they are not part of the header `stdlib.h` as specified in the C standard library.

26.8 [c.math] p7 adds signatures `std::abs(long)` and `std::abs(long long)`, but not in a way that seems to allow `::abs(long)` and `::abs(long long)` to be provided.

I think the right approach here would be to allow `<cstdlib>` to either provide no `::abs` declaration, or to provide all three declarations from namespace `std`, but it should not be permitted to provide only `int abs(int)`. Suggestion:

Change in 17.6.1.2 [headers] p4:

[...]. It is unspecified whether these names [\(including any overloads added in Clauses 18 through 30 and Annex D\)](#) are first declared within the global namespace scope and are then injected into namespace `std` by explicit using-declarations (7.3.3).

[2015-05, Lenexa]

MC: do we need to defer this?

PJP: just need to get my mind around it, already playing dirty games here, my reaction is just do it as it will help C++

STL: this is safe

TP: would be surprising if using `abs` didn't bring in all of the overloads

MC: that's Richard's argument

MC: move to ready

Proposed resolution:

This wording is relative to N3936.

1. Modify 17.6.1.2 [headers] p4 as indicated:

Except as noted in Clauses 18 through 30 and Annex D, the contents of each header *cname* shall be the same as that of the corresponding header *name.h*, as specified in the C standard library (1.2) or the C Unicode TR, as appropriate, as if by inclusion. In the C++ standard library, however, the declarations (except for names which are defined as macros in C) are within namespace scope (3.3.6) of the namespace `std`. It is unspecified whether these names [\(including any overloads added in Clauses 18 through 30 and Annex D\)](#) are first declared within the global namespace scope and are then injected into namespace `std` by explicit using-declarations (7.3.3).

[2384](#). Allocator's `deallocate` function needs better specification

Section: 17.6.3.5 [allocator.requirements] **Status:** [Ready](#) **Submitter:** Daniel Krügler **Opened:** 2014-05-19 **Last modified:** 2015-05-

View other [active issues](#) in [allocator.requirements].

View all other [issues](#) in [allocator.requirements].

View all issues with [Ready](#) status.

Discussion:

According to Table 28, 17.6.3.5 [allocator.requirements], an Allocator's deallocate function is specified as follows:

All n T objects in the area pointed to by p shall be destroyed prior to this call. n shall match the value passed to allocate to obtain this memory. Does not throw exceptions. [Note: p shall not be singular. — *end note*]

This wording is confusing in regard to the following points:

1. This specification does not make clear that the result of an allocate call can only be returned once to the deallocate function. This is much clearer expressed for operator delete (18.6.1.1 [new.delete.single] p12, emphasis mine):

Requires: ptr shall be a null pointer or **its value shall be a value returned by an earlier call to** the (possibly replaced) **operator new(std::size_t) OR operator new(std::size_t, const std::nothrow_t&) which has not been invalidated by an intervening call to operator delete(void*)**.

2. The intended meaning of that wording was to say that deallocate shall accept *every* result value that had been returned by a corresponding call to allocate, this includes also a possible result of a null pointer value, which is possible ("[Note: If $n == 0$, the return value is unspecified. — *end note*]"). Unfortunately the deallocate function uses a non-normative note (" p shall not be singular.") which refers to the fuzzy term *singular*, that is one of the most unclear and misunderstood terms of the library, as pointed out in [1213](#). The occurrence of this term has lead to the possible understanding, that this function would never allow null pointer values. Albeit for allocators the intention had not been to require the support *in general* that a null pointer value can be provided to deallocate (as it is allowed for `std::free` and `operator delete`), the mental model was that *every* returned value of allocate shall be an acceptable argument type of the corresponding deallocate function.

This issue is not intending to enforce a specific meaning of *singular* iterator values, but the assertion is that this note does more harm than good. In addition to wording from operator delete there is no longer any need to obfuscate the normative wording.

[2014-05-24 Alisdair comments]

Now that I am reading it very precisely, there is another mis-stated assumption as a precondition for deallocate:

All n T objects in the area pointed to by p shall be destroyed prior to this call.

This makes a poor assumption that every possible object in the allocated buffer was indeed constructed, but this is often not the case, e.g., a vector that is not filled to capacity. We should require calling the destructor for only those objects actually constructed in the buffer, which may be fewer than n , or even 0.

I wonder if we really require all objects to be destroyed before calling deallocate though. Are we really so concerned about leaking objects that might not manage resources? Should this not be the proper concern of the library managing the objects and memory?

[2014-06-05 Daniel responds and improves wording]

I fully agree with the last comment and I think that this requirement should be removed. We have no such requirements for comparable functions such as operator delete OR return_temporary_buffer(), and this wording seems to be a wording rudiment that exists since C++98.

[2015-05, Lenexa]

Marshall: What do people think about this?

PJP: Sure.

Wakely: Love it.

Marshall: Ready?

Everyone agrees.

Proposed resolution:

This wording is relative to N3936.

1. Change Table 28 ("Allocator requirements") as indicated:

Table 28 — Allocator requirements

Expression	Return type	Assertion/note pre-/post-condition	Default
...			
<code>a.deallocate(p,n)</code>	(not used)	<i>Pre:</i> <code>p</code> shall be a value returned by an earlier call to <code>allocate</code> which has not been invalidated by an intervening call to <code>deallocate</code>. <code>n</code> shall match the value passed to <code>allocate</code> to obtain this memory. All <code>n</code> <code>T</code> objects in the area pointed to by <code>p</code> shall be destroyed prior to this call. <i>Throws:</i> Nothing. <code>n</code> shall match the value passed to <code>allocate</code> to obtain this memory. Does not throw exceptions. [Note: <code>p</code> shall not be singular. — end note]	
...			

[2385](#). `function::assign` allocator argument doesn't make sense

Section: 20.9.12.2 [func.wrap.func] **Status:** [Ready](#) **Submitter:** Pablo Halpern **Opened:** 2014-05-23 **Last modified:** 2015-05-07

View other [active issues](#) in [func.wrap.func].

View all other [issues](#) in [func.wrap.func].

View all issues with [Ready](#) status.

Discussion:

The definition of `function::assign` in N3936 is:

```
template<class F, class A>
void assign(F&& f, const A& a);

Effects: function(allocator_arg, a, std::forward<F>(f)).swap(*this)
```

This definition is flawed in several respects:

1. The interface implies that the intent is to replace the allocator in `*this` with the specified allocator, `a`. Such functionality is unique in the standard and is problematic when creating, e.g. a container of function objects, all using the same allocator.
2. The current definition of `swap()` makes it unclear whether the objects being swapped can have different allocators. The general practice is that allocators must be equal in order to support swap, and this practice is reinforced by the proposed library TS. Thus, the definition of `assign` would have undefined behavior unless the allocator matched the allocator already within function.
3. The general rule for members of `function` is to supply the allocator before the functor, using the `allocator_arg` prefix. Supplying the allocator as a second argument, without the `allocator_arg` prefix is error prone and confusing.

I believe that this ill-conceived interface was introduced in the effort to add allocators to parts of the standard where it had been missing, when we were unpracticed in the right way to accomplish that. Allocators were added to `function` at a time when the allocator model was in flux and it was the first class in the standard after `shared_ptr` to use type-erased allocators, so it is not surprising to see some errors in specification here. (`shared_ptr` is a special case because of its shared semantics, and so is not a good model.)

The main question is whether this member should be specified with better precision or whether it should be deprecated/removed. The only way I can see to give a "reasonable" meaning to the existing interface is to describe it in terms of destroying and re-constructing `*this`:

```
function temp(allocator_arg, a, std::forward<F>(f));
this->~function();
::new(this) function(std::move(temp));
```

(The `temp` variable is needed for exception safety). The ugliness of this specification underscores the ugliness of the concept.

What is the purpose of this member other than to reconstruct the object from scratch, a facility that library classes do not generally provide? Programmers are always free to destroy and re-construct objects — there is no reason why function should make that especially easy.

I propose, therefore, that we make no attempt at giving the current interface a meaningful definition of questionable utility, but simply get rid of it all together. This leaves us with only two questions:

1. Should we deprecate the interface or just remove it?
2. Should we replace it with an `assign(f)` member that doesn't take an allocator?

Of these four combinations of binary answers to the above questions, I think the ones that make the most sense are (remove, no) and (deprecate, yes). The proposed new interface provides nothing that `operator=` does not already provide. However, if the old (deprecated) interface remains, then having the new interface will guide the programmer away from it.

The proposed wording below assumes deprecation. If we choose removal, then there is no wording needed; simply remove the offending declaration and definition.

Previous resolution [SUPERSEDED]:

This wording is relative to N3936.

1. Change class template function synopsis, 20.9.12.2 [func.wrap.func], as indicated:

```
template<class R, class... ArgTypes>
class function<R(ArgTypes...)> {
    [...]
    // 20.9.11.2.2, function modifiers:
    void swap(function&) noexcept;
    template<class F, class A> void assign(F&&, const A&);
    [...]
};
```

2. Change 20.9.12.2.2 [func.wrap.func.mod] as indicated:

```
template<class F, class A> void assign(F&& f, const A& a);

Effects: *this = forward<F>(f); function(allocator_arg, a, std::forward<F>
(f)).swap(*this)
```

3. To deprecation section [depr.function.objects], add the following new sub-clause:

Old `assign` member of polymorphic function wrappers [depr.function.objects.assign]

```
namespace std{
    template<class R, class... ArgTypes>
    class function<R(ArgTypes...)> {
        // remainder unchanged
        template<class F, class A> void assign(F&& f, const A& a);
        [...]
    };
}
```

The two-argument form of `assign` is defined as follows:

```
template<class F, class A> void assign(F&& f, const A& a);
```

Requires: `a` shall be equivalent to the allocator used to construct `*this`.

Effects: `this->assign(forward<F>(f));`

[2015-05, Lenexa]

STL: I would ask, does anybody oppose removing this outright?

Wakely: I don't even have this signature.

Hwrd: And I think this doesn't go far enough, even more should be removed. This is a step in the right direction.

PJP: I'm in favor of removal.

Wakely: We've already got TS1 that has a new function that does it right. We could wait for feedback on that. I think this issue should be taken now.

Marshall: Then the goal will be to move to ready.

Proposed resolution:

This wording is relative to N4431.

1. Change class template function synopsis, 20.9.12.2 [func.wrap.func], as indicated:

```
template<class R, class... ArgTypes>
class function<R(ArgTypes...)> {
    [...]
    // 20.9.12.2.2, function modifiers:
    void swap(function&) noexcept;
    template<class F, class A> void assign(F&&, const A&);
    [...]
};
```

2. Change 20.9.12.2.2 [func.wrap.func.mod] as indicated:

```
template<class F, class A>
void assign(F&& f, const A& a);

-2- Effects: function(allocator_arg, a, std::forward<F>(f)).swap(*this)
```

2435. reference_wrapper::operator()'s Remark should be deleted

Section: 20.9.4.4 [refwrap.invoke] **Status:** [Ready](#) **Submitter:** Stephan T. Lavavej **Opened:** 2014-10-01 **Last modified:** 2015-05-08

View all issues with [Ready](#) status.

Discussion:

20.9.4.4 [refwrap.invoke]/2 is no longer useful. (It was originally TR1 2.1.2.4 [tr.util.refwrp.invoke]/2.) First, we already have the As If Rule (1.9 [intro.execution]/1) and the STL Implementers Can Be Sneaky Rule (17.6.5.5 [member.functions]). Second, with variadic templates and other C++11/14 tech, this can be implemented exactly as depicted.

[2015-05, Lenexa]

DK: I don't see a defect here

STL: the issue is that the standard is overly verbose, we don't need this sentence. It's redundant.

MC: does anyone think this paragraph has value?

JW: it has negative value. reading it makes me wonder if there's some reason I would want to provide a set of overloaded functions, maybe there's some problem with doing it the obvious way that I'm not clever enough to see.

Move to Ready status: 8 in favor, none against.

Proposed resolution:

This wording is relative to N3936.

1. Change 20.9.4.4 [refwrap.invoke] p2 as depicted:

```
template <class... ArgTypes>
result_of_t<T&(ArgTypes&&...)>
operator()(ArgTypes&&... args) const;

-1- Returns: INVOKE(get(), std::forward<ArgTypes>(args)...). (20.9.2)
```

```
-2- Remark: operator() is described for exposition only. Implementations are not required to provide
an actual reference_wrapper::operator(). Implementations are permitted to support reference_wrapper
function invocation through multiple overloaded operators or through other means.
```

2447. Allocators and volatile-qualified value types

Section: 17.6.3.5 [allocator.requirements] **Status:** [Ready](#) **Submitter:** Daniel Krüglér **Opened:** 2014-10-16 **Last modified:** 2015-09-25

View other [active issues](#) in [allocator.requirements].

View all other [issues](#) in [allocator.requirements].

View all issues with [Ready](#) status.

Discussion:

According to Table 27 — "Descriptive variable definitions" which is used to define the symbols used in the allocator requirements table within 17.6.3.5 [allocator.requirements] we have the following constraints for the types `τ`, `u`, `c`:

any non-const object type (3.9)

This wording can be read to allow instead a `volatile`-qualified value type such as `volatile int`.

The nearest-by way of fixing this would be to add "non-volatile" as additional constraint to this table row.

Another choice would be to think of requiring that allocators must be capable to handle any `cv`-qualified value types. This would make all currently existing allocators non-conforming that can't handle `cv`-qualified value types, so I'm not suggesting to follow that route.

A less radical step would be to allow `cv`-qualified types just for `c` (which is used to specify the functions `construct` and `destroy` and where does not even exist any requirement that `c` actually is related to the value type of the allocator at all). This seemingly extension would be harmless because as of p8 of the same sub-clause "An allocator may constrain the types on which it can be instantiated and the arguments for which its `construct` member may be called."

This differs from the requirements imposed on the types `τ` and `u` which both refer to value types of allocators.

The proposed wording attempts to separate the two classes of requirements.

Previous resolution [SUPERSEDED]:

This wording is relative to N4140.

- 1. Change 17.6.3.5 [allocator.requirements], Table 27 — "Descriptive variable definitions", as indicated:

Table 27 — Descriptive variable definitions	
Variable	Definition
<code>τ</code> , <code>u</code> , <code>c</code>	any non-const <u>const and non-volatile</u> object type (3.9)
<code>c</code>	<u>any object type</u>
...	

- 2. Change 17.6.3.5 [allocator.requirements] p8 as indicated: (This wording change is intended to fix an obvious asymmetry between `construct` and `destroy` which I believe is not intended)

-8- An allocator may constrain the types on which it can be instantiated and the arguments for which its `construct` or destroy members may be called. If a type cannot be used with a particular allocator, the allocator class or the call to `construct` or destroy may fail to instantiate.

[2014-11, Urbana]

JW: say "cv-unqualified" instead?
JW: very nervous about allowing `construct` on `const`-types, because of the cast to `(non-const) void*`
MA: should we just make the minimal fix?
STL: don't break `c` out for special treatment
New proposed resolution: just change "non-const" to "cv-unqualified". Keep addition of `destroy` later.

[2015-02 Cologne]

GR: It makes me nervous that someone at some point decided to not add "non-volatile".
AM: That was over ten years ago. It was a deliberate, minuted choice to support `volatile`. We are now reversing that decision. It would be good to poll our vendors, none of which are in the room. This is a bit more work than we expect of a P0 issue.
VV: `libstdc++` and `libc++` seem to support `volatile` template parameters for the standard allocator.
AM: To clarify, the proposed resolution here would remove the requirement to support `volatile`. Implementations could still choose to support `volatile`.
DK: I'm happy to drop this and open a new issue in regard to the `destroy` member specification.
AM: I just think this is harder than a P0. Let's reprioritize.

[2015-04-01 Daniel comments]

The less controversial part of the issue related to constraints imposed on `destroy` has be handed over to the new issue [2470](#).

Proposed resolution:

This wording is relative to N4431.

1. Change 17.6.3.5 [allocator.requirements], Table 27 — "Descriptive variable definitions", as indicated:

Table 27 — Descriptive variable definitions

Variable	Definition
T, U, C	any non-const cv-unqualified object type (3.9)
...	

[2462](#). `std::ios_base::failure` is overspecified

Section: 27.5.3 [ios.base], 27.5.3.1.1 [ios::failure] **Status:** [Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2014-12-15 **Last modified:** 2015-05-22

View all other [issues](#) in [ios.base].

View all issues with [Ready](#) status.

Discussion:

27.5.3 [ios.base] defines `ios_base::failure` as a nested class:

```
namespace std {
    class ios_base {
    public:
        class failure;
        [...]
    };
    [...]
}
```

This means it is valid to use an elaborated-type-specifier to refer to `ios_base::failure`:

```
using F = class std::ios_base::failure;
throw F();
```

Therefore implementations are not permitted to define `ios_base::failure` as a typedef e.g.

```
class ios_base {
public:
    #if __cplusplus < 201103L
        class failure_cxx03 : public exception {...};
        typedef failure_cxx03 failure;
    #else
        class failure_cxx11 : public system_error {...};
        typedef failure_cxx11 failure;
    #endif
    [...]
};
```

This constrains implementations, making it harder to manage the ABI change to `ios_base::failure` between C++03 and C++11.

[2015-05-06 Lenexa: Move to Ready]

JW: the issue is that users are currently allowed to write "class failure" with an elaborated-type-specifier and it must be well-formed, I want the freedom to make that type a typedef, so they can't necessarily use an elaborated-type-specifier (which there is no good reason to use anyway)

JW: ideally I'd like this everywhere for all nested classes, but that's a paper not an issue, I only need this type fixed right now.

RD: is a synonym the same as an alias?

JW: `dcl.typedef` says a typedef introduces a synonym for another type, so I think this is the right way to say this

JW: I already shipped this last month

PJP: we're going to have to break ABIs again some time, we need all the wiggle room we can get to make that easier. This helps.

MC: do we want this at all? Ready?

9 in favor, none oppose or abstaining

Proposed resolution:

This wording is relative to N4296.

1. Change the synopsis in 27.5.3 [ios.base] as indicated:

```
namespace std {  
    class ios_base {  
    public:  
        class failure; // see below  
        [...]  
    };  
    [...]  
};
```

2. Change 27.5.3 [ios.base] paragraph 1:

ios_base defines several member types:

- a ~~class failure~~ type failure, defined as either a class derived from system_error or a synonym for a class derived from system_error;

3. Change 27.5.3.1.1 [ios::failure] paragraph 1:

-1- An implementation is permitted to define ios_base::failure as a synonym for a class with equivalent functionality to class ios_base::failure shown in this subclause. [Note: When ios_base::failure is a synonym for another type it shall provide a nested type failure, to emulate the injected class name. — end note] The class failure defines the base class for the types of all objects thrown as exceptions, by functions in the iostreams library, to report errors detected during stream buffer operations.

2466. allocator_traits::max_size() default behavior is incorrect

Section: 17.6.3.5 [allocator.requirements], 20.7.8.2 [allocator.traits.members] **Status:** [Ready](#) **Submitter:** Howard Hinnant
Opened: 2015-01-17 **Last modified:** 2015-05-08

View other [active issues](#) in [allocator.requirements].

View all other [issues](#) in [allocator.requirements].

View all issues with [Ready](#) status.

Discussion:

Table 28 — "Allocator requirements" says that default behavior for a.max_size() is numeric_limits<size_type>::max(). And this is consistent with the matching statement for allocator_traits in 20.7.8.2 [allocator.traits.members]/p7:

```
static size_type max_size(const Alloc& a) noexcept;
```

Returns: a.max_size() if that expression is well-formed; otherwise, numeric_limits<size_type>::max().

However, when allocating memory, an allocator must allocate $n \times \text{sizeof}(\text{value_type})$ bytes, for example:

```
value_type*  
allocate(std::size_t n)  
{  
    return static_cast<value_type*> (::operator new (n * sizeof(value_type)));  
}
```

When $n == \text{numeric_limits}<\text{size_type}>::\text{max}()$, $n * \text{sizeof}(\text{value_type})$ is *guaranteed* to overflow except when $\text{sizeof}(\text{value_type}) == 1$.

A more useful default would be $\text{numeric_limits}<\text{size_type}>::\text{max}() / \text{sizeof}(\text{value_type})$.

[2015-05, Lenexa]

Marshall: Is this the right solution?

PJP: I think it's gilding the lily.

STL: I think this is right, and it doesn't interact with the incomplete container stuff because it's in a member function.
Marshall: Objections to this?
STL: Spaces around binary operators.
Hwrd: It's completely wrong without spaces.
Marshall: All in favor of Ready?
Lots.

Proposed resolution:

This wording is relative to N4296.

- 1. Change 17.6.3.5 [allocator.requirements], Table 28 — "Allocator requirements", as indicated:

Table 28 — Allocator requirements			
Expression	Return type	Assertion/note pre-/post-condition	Default
...			
a.max_size()	X::size_type	the largest value that can meaningfully be passed to X::allocate()	numeric_limits<size_type>::max() /sizeof(value_type)
...			

- 2. Change 20.7.8.2 [allocator.traits.members]/p7 as indicated:

```
static size_type max_size(const Alloc& a) noexcept;
```

Returns: a.max_size() if that expression is well-formed; otherwise,
numeric_limits<size_type>::max() /sizeof(value_type).

2469. Wrong specification of Requires clause of operator[] for map and unordered_map

Section: 23.4.4.3 [map.access], 23.5.4.3 [unord.map.elem] **Status:** [Ready](#) **Submitter:** Tomasz Kamiński **Opened:** 2015-01-21 **Last modified:** 2015-05-08

View all other [issues](#) in [map.access].

View all issues with [Ready](#) status.

Discussion:

The "Requires:" clause for the operator[] for the unordered_map and map, are defining separate requirements for insertability into container of mapped_type and key_type.

23.4.4.3 [map.access] p2: // T& operator[](const key_type& x);

Requires: key_type shall be CopyInsertable and mapped_type shall be DefaultInsertable into *this.

23.4.4.3 [map.access] p6: // T& operator[](key_type&& x)

Requires: mapped_type shall be DefaultInsertable into *this.

23.5.4.3 [unord.map.elem] p1: // mapped_type& operator[](const key_type& k); mapped_type& operator[](key_type&& k);

Requires: mapped_type shall be DefaultInsertable into *this. For the first operator, key_type shall be CopyInsertable into *this. For the second operator, key_type shall be MoveConstructible.

Definition of the appropriate requirements: 23.2.1 [container.requirements.general] p15.

T is DefaultInsertable into X means that the following expression is well-formed: //p15.1

```
allocator_traits<A>::construct(m, p)
```

T is MoveInsertable into X means that the following expression is well-formed: //p15.3

```
allocator_traits<A>::construct(m, p, rv)
```

T is CopyInsertable into X means that, in addition to T being MoveInsertable into X, the following expression is well-formed: //p15.4

```
allocator_traits<A>::construct(m, p, v)
```

In the context of above definition the requirement "key_type shall be CopyInsertable into *this" would mean that the key element of the pair<const key_type, mapped_type> (value_type of the map) should be constructed using separate call to the construct method, the same applies for the mapped_type. Such behavior is explicitly prohibited by 23.2.1 [container.requirements.general] p3.

For the components affected by this sub-clause that declare an allocator_type, objects stored in these components shall be constructed using the allocator_traits<allocator_type>::construct function and destroyed using the allocator_traits<allocator_type>::destroy function (20.7.8.2). These functions are called only for the container's element type, not for internal types used by the container.

It clearly states that element_type of the map, must be constructed using allocator for value type, which disallows using of separate construction of first and second element, regardless of the fact if it can be actually performed without causing undefined behavior.

That means that the MoveInsertable and similar requirements may only be expressed in terms of value_type, not its members types.

[2015-02 Cologne]

This issue is related to [2464](#).

GR: Effects should say "returns ...". DK: Or just have a Returns clause? MC: A Returns clause is a directive to implementers.

TK/DK: This PR fails to address the requirements about which it complained in the first place. DK: I can reword this. TK can help.

[2015-03-29, Daniel provides improved wording]

The revised wording fixes the proper usage of the magic "Equivalent to" wording, which automatically induces *Requires*:, *Returns*:, and *Complexity*: elements (and possibly more). This allows us to strike all the remaining elements, because they fall out from the semantics of the wording defined by [2464](#). In particular it is important to realize that the wording form

```
value_type shall be EmplaceConstructible into map from piecewise_construct, forward_as_tuple(k),
forward_as_tuple(forward<Args>(args)...) )
```

degenerates for the empty pack expansion args to:

```
value_type shall be EmplaceConstructible into map from piecewise_construct, forward_as_tuple(k), forward_as_tuple()
```

which again means that such a pair construction (assuming std::allocator) would copy k into member first and would value-initialize member second.

Previous resolution [SUPERSEDED]:

This wording is relative to N4296.

Accept resolution of the issue [2464](#) and define operator[] as follows (This would also address issue [2274](#)):

1. Change 23.4.4.3 [map.access] as indicated:

```
T& operator[](const key_type& x);
```

-1- *Effects*: ~~If there is no key equivalent to x in the map, inserts value_type(x, T()) into the map~~ **Equivalent to: try_emplace(x).first->second.**

```
[...]
```

```
T& operator[](key_type&& x);
```

-5- *Effects*: ~~If there is no key equivalent to x in the map, inserts value_type(std::move(x), T()) into the map~~ **Equivalent to: try_emplace(move(x)).first->second.**

2. Change 23.5.4.3 [unord.map.elem] as indicated:

```
mapped_type& operator[](const key_type& k);
mapped_type& operator[](key_type&& k);
```

```
[...]
```

~~-2- Effects: If the unordered_map does not already contain an element whose key is equivalent to k, the first operator inserts the value value_type(k, mapped_type()) and the second operator inserts the value value_type(std::move(k), mapped_type())~~For the first operator, equivalent to: try_emplace(k).first->second; for the second operator, equivalent to: try_emplace(move(k)).first->second.

Previous resolution [SUPERSEDED]:

This wording is relative to N4296.

Accept resolution of the issue [2464](#) and define operator[] as follows (This would also address issue [2274](#)):

1. Change 23.4.4.3 [map.access] as indicated:

```
T& operator[](const key_type& x);
```

~~-1- Effects: If there is no key equivalent to x in the map, inserts value_type(x, T()) into the map.~~Equivalent to: return try_emplace(x).first->second;

~~-2- Requires: key_type shall be CopyInsertable and mapped_type shall be DefaultInsertable into *this.~~

~~-3- Returns: A reference to the mapped_type corresponding to x in *this.~~

~~-4- Complexity: Logarithmic.~~

```
T& operator[](key_type&& x);
```

~~-5- Effects: If there is no key equivalent to x in the map, inserts value_type(std::move(x), T()) into the map.~~Equivalent to: return try_emplace(move(x)).first->second;

~~-6- Requires: mapped_type shall be DefaultInsertable into *this.~~

~~-7- Returns: A reference to the mapped_type corresponding to x in *this.~~

~~-8- Complexity: Logarithmic.~~

2. Change 23.5.4.3 [unord.map.elem] as indicated:

```
mapped_type& operator[](const key_type& k);  
mapped_type& operator[](key_type&& k);
```

~~-1- Requires: mapped_type shall be DefaultInsertable into *this. For the first operator, key_type shall be CopyInsertable into *this. For the second operator, key_type shall be MoveConstructible.~~

~~-2- Effects: If the unordered_map does not already contain an element whose key is equivalent to k, the first operator inserts the value value_type(k, mapped_type()) and the second operator inserts the value value_type(std::move(k), mapped_type())~~For the first operator, equivalent to:

```
return try_emplace(k).first->second;
```

for the second operator, equivalent to:

```
return try_emplace(move(k)).first->second;
```

~~-3- Returns: A reference to x.second, where x is the (unique) element whose key is equivalent to k.~~

~~-4- Complexity: Average case $\mathcal{O}(1)$, worst case $\mathcal{O}(\text{size}())$.~~

Proposed resolution:

This wording is relative to N4431.

Accept resolution of the issue [2464](#) and define operator[] as follows (This would also address issue [2274](#)):

1. Change 23.4.4.3 [map.access] as indicated:

```
T& operator[](const key_type& x);
```

- 1- *Effects:* If there is no key equivalent to x in the map, inserts $\text{value_type}(x, T())$ into the map. **Equivalent to: `return try_emplace(x).first->second;`**
- 2- *Requires:* key_type shall be `CopyInsertable` and mapped_type shall be `DefaultInsertable` into $*\text{this}$.
- 3- *Returns:* A reference to the mapped_type corresponding to x in $*\text{this}$.
- 4- *Complexity:* Logarithmic.

```
T& operator[](key_type&& x);
```

- 5- *Effects:* If there is no key equivalent to x in the map, inserts $\text{value_type}(\text{std}::\text{move}(x), T())$ into the map. **Equivalent to: `return try_emplace(move(x)).first->second;`**
- 6- *Requires:* mapped_type shall be `DefaultInsertable` into $*\text{this}$.
- 7- *Returns:* A reference to the mapped_type corresponding to x in $*\text{this}$.
- 8- *Complexity:* Logarithmic.

2. Change 23.5.4.3 [unord.map.elem] as indicated:

```
mapped_type& operator[](const key_type& k);  
mapped_type& operator[](key_type&& k);
```

- 1- *Requires:* mapped_type shall be `DefaultInsertable` into $*\text{this}$. For the first operator, key_type shall be `CopyInsertable` into $*\text{this}$. For the second operator, key_type shall be `MoveConstructible`.
- 2- *Effects:* **Equivalent to `return try_emplace(k).first->second;`** If the unordered_map does not already contain an element whose key is equivalent to k , the first operator inserts the value $\text{value_type}(k, \text{mapped_type}())$ and the second operator inserts the value $\text{value_type}(\text{std}::\text{move}(k), \text{mapped_type}())$.
- 3- *Returns:* A reference to $x.\text{second}$, where x is the (unique) element whose key is equivalent to k .
- 4- *Complexity:* Average case $O(1)$, worst case $O(\text{size}())$.

```
mapped_type& operator[](key_type&& k);
```

- ?- *Effects:* Equivalent to `return try_emplace(move(k)).first->second;`**

2473. basic_filebuf's relation to C FILE semantics

Section: 27.9.1.5 [filebuf.virtuals] **Status:** [Ready](#) **Submitter:** Aaron Ballman **Opened:** 2015-02-09 **Last modified:** 2015-10-20

View all other [issues](#) in [filebuf.virtuals].

View all issues with [Ready](#) status.

Discussion:

The restrictions on reading and writing a sequence controlled by an object of class `basic_filebuf<charT, traits>` are the same as for reading and writing with the Standard C library `FILES`. One of the restrictions placed by C is on the behavior of a stream that is opened for input and output. See the C99 standard, 7.19.5.3p6 for more details, but the gist is that when opened in update mode, reads and writes must have an intervening file positioning or flushing call to not trigger UB.

27.9.1.5 [filebuf.virtuals] p13 specifies that `basic_filebuf::seekoff()` calls `std::fseek()`. However, there is no mention of `std::fseek()` in `basic_filebuf::seekpos()`, and no mention of `std::fflush()` in `basic_filebuf::sync()`, which seem like an oversight.

Previous resolution [SUPERSEDED]:

This wording is relative to N4296.

1. Change 27.9.1.5 [filebuf.virtuals] p16 as follows [Editorial note: A footnote referring to `fseek` is not needed, because this is already covered by the existing footnote 334]:

-16- Alters the file position, if possible, to correspond to the position stored in `sp` (as described below). Altering the file position performs as follows:

1. if `(om & ios_base::out) != 0`, then update the output sequence and write any unshift sequence;
2. set the file position to `sp` as if by calling `std::fseek(file, sp, SEEK_SET)`;
3. if `(om & ios_base::in) != 0`, then update the input sequence;

where `om` is the open mode passed to the last call to `open()`. The operation fails if `is_open()` returns false.

2. Change 27.9.1.5 [filebuf.virtuals] p19 as follows and add a new footnote that mimics comparable footnotes in 27.9.1.4 [filebuf.members] and 27.9.1.5 [filebuf.virtuals]:

-19- *Effects*: If a put area exists, calls `filebuf::overflow` to write the characters to the file, then flushes the file as if by calling `std::fflush(file)` [Footnote: The function signature `fflush(FILE*)` is declared in `<stdio>` (27.9.2)]. If a get area exists, the effect is implementation-defined.

[2015-05, Lenexa]

Aaron provides improved wording by removing the params from `std::fseek()` due to the concerns regarding the parameters on systems where `fseek` uses 32-bit parameters.

Second wording improvement, replacing the new one see below. It

1. drops the `std::`
2. drops the footnote for `fflush`
3. replaces `fseek` with `fsetpos`

Previous resolution [SUPERSEDED]:

This wording is relative to N4431.

1. Change 27.9.1.5 [filebuf.virtuals] p16 as follows [Editorial note: A footnote referring to `fseek` is not needed, because this is already covered by the existing footnote 334]:

-16- Alters the file position, if possible, to correspond to the position stored in `sp` (as described below). Altering the file position performs as follows:

1. if `(om & ios_base::out) != 0`, then update the output sequence and write any unshift sequence;
2. set the file position to `sp` as if by a call to `std::fseek`;
3. if `(om & ios_base::in) != 0`, then update the input sequence;

where `om` is the open mode passed to the last call to `open()`. The operation fails if `is_open()` returns false.

2. Change 27.9.1.5 [filebuf.virtuals] p19 as follows and add a new footnote that mimics comparable footnotes in 27.9.1.4 [filebuf.members] and 27.9.1.5 [filebuf.virtuals]:

-19- *Effects*: If a put area exists, calls `filebuf::overflow` to write the characters to the file, then flushes the file as if by calling `std::fflush(file)` [Footnote: The function signature `fflush(FILE*)` is declared in `<stdio>` (27.9.2)]. If a get area exists, the effect is implementation-defined.

Proposed resolution:

This wording is relative to N4431.

1. Change 27.9.1.5 [filebuf.virtuals] p16 as follows:

-16- Alters the file position, if possible, to correspond to the position stored in `sp` (as described below). Altering

the file position performs as follows:

1. if (om & ios_base::out) != 0, then update the output sequence and write any unshift sequence;
2. set the file position to sp **as if by a call to `fsetpos`**;
3. if (om & ios_base::in) != 0, then update the input sequence;

where om is the open mode passed to the last call to `open()`. The operation fails if `is_open()` returns false.

2. Change 27.9.1.5 [filebuf.virtuals] p19 as follows:

-19- *Effects*: If a put area exists, calls `filebuf::overflow` to write the characters to the file, **then flushes the file as if by calling `fflush(file)`**. If a get area exists, the effect is implementation-defined.

2476. `scoped_allocator_adaptor` is not assignable

Section: 20.13.1 [allocator.adaptor.syn] **Status:** [Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2015-03-02 **Last modified:** 2015-05-08

View all issues with [Ready](#) status.

Discussion:

The class definition in 20.13.1 [allocator.adaptor.syn] declares a move constructor, which means that the copy assignment operator is defined as deleted, and no move assignment operator is declared.

This means a `scoped_allocator_adaptor` is not assignable, and a container using `scoped_allocator_adaptor<A...>` may not be `CopyAssignable` or `MoveAssignable` (depending on the `propagate_on_container_XXXX_assignment` traits of the outer and inner allocator types).

[2015-04-03 Howard comments]

If the contained allocators are not assignable, I think we need the ability of `= default` to automatically become `= delete`. My concern is that `is_copy_assignable<scoped_allocator_adaptor<CustomAllocator>>::value` get the right answer for both cases:

1. `is_copy_assignable<CustomAllocator>::value` is true.
2. `is_copy_assignable<CustomAllocator>::value` is false.

If we allow the vendor to declare and provide the copy assignment operator, the chance of getting #2 correct goes to zero.

Previous resolution [SUPERSEDED]:

This wording is relative to N4296.

1. Add to the synopsis in 20.13.1 [allocator.adaptor.syn]/1 *[Editorial remark: The proposed wording does not explicitly specify the semantics of the added copy/move assignment operators, based on 17.5.2.2 [functions.within.classes] p1, which says:*

"For the sake of exposition, Clauses 18 through 30 and Annex D do not describe copy/move constructors, assignment operators, or (non-virtual) destructors with the same apparent semantics as those that can be generated by default (12.1, 12.4, 12.8)."

— *end remark*]:

```
[...]
template <class OuterA2>
    scoped_allocator_adaptor(
        scoped_allocator_adaptor<OuterA2, InnerAllocs...>&& other) noexcept;

    scoped_allocator_adaptor& operator=(const scoped_allocator_adaptor&);
    scoped_allocator_adaptor& operator=(scoped_allocator_adaptor&&);

    ~scoped_allocator_adaptor();
[...]
```

[2015-05, Lenexa]

Move to Immediate.

Proposed resolution:

This wording is relative to N4296.

1. Add to the synopsis in 20.13.1 [allocator.adaptor.syn]/1:

```
[...]
template <class OuterA2>
    scoped_allocator_adaptor(
        scoped_allocator_adaptor<OuterA2, InnerAllocs...&& other> noexcept;

    scoped_allocator_adaptor& operator=(const scoped_allocator_adaptor&) = default;
    scoped_allocator_adaptor& operator=(scoped_allocator_adaptor&&) = default;

~scoped_allocator_adaptor();
[...]
```

2477. Inconsistency of wordings in `std::vector::erase()` and `std::deque::erase()`

Section: 23.3.3.4 [deque.modifiers], 23.3.6.5 [vector.modifiers] **Status:** [Ready](#) **Submitter:** Anton Savin **Opened:** 2015-03-03 **Last modified:** 2015-05-08

View all other [issues](#) in [deque.modifiers].

View all issues with [Ready](#) status.

Discussion:

In the latest draft N4296, and in all drafts up to at least N3337:

23.3.3.4 [deque.modifiers]/5 (regarding `deque::erase()`):

Complexity: The number of calls to the destructor is the same as the number of elements erased, but the number of calls to the **assignment operator** is no more than the lesser of the number of elements before the erased elements and the number of elements after the erased elements.

23.3.6.5 [vector.modifiers]/4 (regarding `vector::erase()`):

Complexity: The destructor of τ is called the number of times equal to the number of the elements erased, but the **move assignment operator** of τ is called the number of times equal to the number of elements in the vector after the erased elements.

Is there any reason for explicit mentioning of move assignment for `std::vector::erase()`? Shouldn't these two wordings be the same with this regard?

Also, for `std::deque`, it's not clear from the text which destructors and assignment operators are called.

[2015-05, Lenexa]

Move to Immediate.

Proposed resolution:

This wording is relative to N4296.

1. Change 23.3.3.4 [deque.modifiers]/5 to:

-5- *Complexity:* The number of calls to the destructor **of τ** is the same as the number of elements erased, but the number of calls to the assignment operator **of τ** is no more than the lesser of the number of elements before the erased elements and the number of elements after the erased elements.

2. Change 23.3.6.5 [vector.modifiers]/4 to:

-4- *Complexity:* The destructor of τ is called the number of times equal to the number of the elements erased, but the **move** assignment operator of τ is called the number of times equal to the number of elements in the vector after the erased elements.

[2483](#). `throw_with_nested()` should use `is_final`

Section: 18.8.6 [except.nested] **Status:** [Ready](#) **Submitter:** Stephan T. Lavavej **Opened:** 2015-03-27 **Last modified:** 2015-05-08

View other [active issues](#) in [except.nested].

View all other [issues](#) in [except.nested].

View all issues with [Ready](#) status.

Discussion:

When [N2559](#) was voted into the Working Paper, it said "This function template must take special case to handle non-class types, unions and `[[final]]` classes that cannot be derived from, and [...]". However, its Standardese didn't handle `final` classes, and this was never revisited. Now that we have `is_final`, we can achieve this proposal's original intention.

Additionally, we need to handle the case where `U` is `nested_exception` itself. `is_base_of`'s wording handles this and ignores cv-qualifiers. (Note that `is_class` detects "non-union class type".)

[2015-05, Lenexa]

STL, MC and JW already do this

MC: move to Ready, bring to motion on Friday

7 in favor, none opposed

Proposed resolution:

This wording is relative to N4296.

1. Change 18.8.6 [except.nested] as depicted:

```
template <class T> [[noreturn]] void throw_with_nested(T&& t);
```

-6- Let `U` be `remove_reference_t<T>`.

-7- Requires: `U` shall be CopyConstructible.

-8- Throws: if `U` is a non-union class type not derived from `nested_exception` `is_class<U>::value && !is_final<U>::value && !is_base_of<nested_exception, U>::value` is true, an exception of unspecified type that is publicly derived from both `U` and `nested_exception` and constructed from `std::forward<T>(t)`, otherwise `std::forward<T>(t)`.

[2484](#). `rethrow_if_nested()` is doubly unimplementable

Section: 18.8.6 [except.nested] **Status:** [Ready](#) **Submitter:** Stephan T. Lavavej **Opened:** 2015-03-27 **Last modified:** 2015-05-08

View other [active issues](#) in [except.nested].

View all other [issues](#) in [except.nested].

View all issues with [Ready](#) status.

Discussion:

`rethrow_if_nested()` wants to determine "If the dynamic type of `e` is publicly and unambiguously derived from `nested_exception`", but 5.2.7 [expr.dynamic.cast] specifies that `dynamic_cast` has a couple of limitations.

First, nonpolymorphic inputs. These could be non-classes, or nonpolymorphic classes. The Standardese handles non-classes, although implementers need special logic. (If `E` is `int`, the dynamic type can't possibly derive from `nested_exception`. Implementers need to detect this and avoid `dynamic_cast`, which would be ill-formed due to 5.2.7 [expr.dynamic.cast]/2.) The Standardese is defective when `E` is a nonpolymorphic class. Consider the following example:

```
struct Nonpolymorphic { };
struct MostDerived : Nonpolymorphic, nested_exception { };
MostDerived md;
const Nonpolymorphic& np = md;
rethrow_if_nested(np);
```

According to 1.3.8 [defs.dynamic.type], the dynamic type of `np` is `MostDerived`. However, it's physically impossible to discover

this, and attempting to do so will lead to an ill-formed `dynamic_cast` (5.2.7 [expr.dynamic.cast]/6). The Standardese must be changed to say that if `E` is nonpolymorphic, nothing happens.

Second, statically good but dynamically bad inputs. Consider the following example:

```
struct Nested1 : nested_exception { };
struct Nested2 : nested_exception { };
struct Ambiguous : Nested1, Nested2 { };
Ambiguous amb;
const Nested1& n1 = amb;
rethrow_if_nested(n1);
```

Here, the static type `Nested1` is good (i.e. publicly and unambiguously derived from `nested_exception`), but the dynamic type `Ambiguous` is bad. The Standardese currently says that we have to detect the dynamic badness, but `dynamic_cast` won't let us. 5.2.7 [expr.dynamic.cast]/3 and /5 are special cases (identity-casting and upcasting, respectively) that activate before the "run-time check" behavior that we want (/6 and below). Statically good inputs succeed (regardless of the dynamic type) and statically bad inputs are ill-formed (implementers must use type traits to avoid this).

It might be possible to implement this with clever trickery involving virtual base classes, but implementers shouldn't be asked to do that. It would definitely be possible to implement this with a compiler hook (a special version of `dynamic_cast`), but implementers shouldn't be asked to do so much work for such an unimportant case. (This case is pathological because the usual way of adding `nested_exception` inheritance is `throw_with_nested()`, which avoids creating bad inheritance.) The Standardese should be changed to say that statically good inputs are considered good.

Finally, we want `is_base_of`'s "base class or same class" semantics. If the static type is `nested_exception`, we have to consider it good due to `dynamic_cast`'s identity-casting behavior. And if the dynamic type is `nested_exception`, it is definitely good.

[2015-05, Lenexa]

WB: so the `is_polymorphic` trait must be used?

STL and JW: yes, that must be used to decide whether to try using `dynamic_cast` or not.

JW: I'd already made this fix in our implementation

STL: the harder case also involves `dynamic_cast`. should not try using `dynamic_cast` if we can statically detect it is OK, doing the `dynamic_cast` might fail.

STL: finally, want "is the same or derived from" behaviour of `is_base_of`

WB: should there be an "else no effect" at the end? We have "Otherwise, if ..." and nothing saying what if the condition is false.

TP I agree.

MC: move to Ready and bring to motion on Friday

7 in favor, none opposed

Proposed resolution:

This wording is relative to N4296.

1. Change 18.8.6 [except.nested] as depicted:

```
template <class E> void rethrow_if_nested(const E& e);
```

-9- *Effects:* If `e` is not a polymorphic class type, there is no effect. Otherwise, if the static type or the dynamic type of `e` is `nested_exception` or is publicly and unambiguously derived from `nested_exception`, calls `dynamic_cast<const nested_exception&>(e).rethrow_nested()`.

2485. `get()` should be overloaded for `const tuple&&`

Section: 20.4.2.6 [tuple.elem] **Status:** [Ready](#) **Submitter:** Stephan T. Lavavej **Opened:** 2015-03-27 **Last modified:** 2015-05-08

View all other [issues](#) in [tuple.elem].

View all issues with [Ready](#) status.

Discussion:

`const rvalues` are weird, but they're part of the type system. Consider the following code:

```
#include <functional>
#include <string>
#include <tuple>

using namespace std;
```

```

string str1() { return "one"; }
const string str2() { return "two"; }
tuple<string> tup3() { return make_tuple("three"); }
const tuple<string> tup4() { return make_tuple("four"); }

int main() {
    // cref(str1()); // BAD, properly rejected
    // cref(str2()); // BAD, properly rejected
    // cref(get<0>(tup3())); // BAD, properly rejected
    cref(get<0>(tup4())); // BAD, but improperly accepted!
}

```

As `tuple` is a fundamental building block (and the only convenient way to have variadic data members), it should not open a hole in the type system. `get()` should imitate 5.2.5 [expr.ref]'s rules for accessing data members. (This is especially true for `pair`, where both `get<0>()` and `.first` are available.)

While we're in the neighborhood, we can dramatically simplify the wording here. All we need to do is follow 20.4.2.6 [tuple.elem]/9's example of saying "Returns: A reference to STUFF", and allow implementers to figure out how to achieve that with the given return types.

[2015-05, Lenexa]

TP: for the existing overloads there's no change to the code, just descriptions?

STL: right.

JW: I love it

MC: in favor of moving to Ready and bringing up for vote on Friday

7 in favor, none opposed

Proposed resolution:

This wording is relative to N4296.

1. Change 20.2 [utility]/2 "Header <utility> synopsis" as depicted:

```

[...]
template<size_t I, class T1, class T2>
constexpr tuple_element_t<I, pair<T1, T2>>&
    get(pair<T1, T2>&) noexcept;
template<size_t I, class T1, class T2>
constexpr tuple_element_t<I, pair<T1, T2>>&&
    get(pair<T1, T2>&&) noexcept;
template<size_t I, class T1, class T2>
constexpr const tuple_element_t<I, pair<T1, T2>>&
    get(const pair<T1, T2>&) noexcept;
template<size_t I, class T1, class T2>
constexpr const tuple_element_t<I, pair<T1, T2>>&&
    get(const pair<T1, T2>&&) noexcept;
template <class T, class U>
constexpr T& get(pair<T, U>& p) noexcept;
template <class T, class U>
constexpr const T& get(const pair<T, U>& p) noexcept;
template <class T, class U>
constexpr T&& get(pair<T, U>&& p) noexcept;
template <class T, class U>
constexpr const T&& get(const pair<T, U>&& p) noexcept;
template <class T, class U>
constexpr T& get(pair<U, T>& p) noexcept;
template <class T, class U>
constexpr const T& get(const pair<U, T>& p) noexcept;
template <class T, class U>
constexpr T&& get(pair<U, T>&& p) noexcept;
template <class T, class U>
constexpr const T&& get(const pair<U, T>&& p) noexcept;
[...]

```

2. Change 20.4.1 [tuple.general]/2 "Header <tuple> synopsis" as depicted:

```

[...]
// 20.4.2.6, element access:
template <size_t I, class... Types>
constexpr tuple_element_t<I, tuple<Types...>>&
    get(tuple<Types...>&) noexcept;
template <size_t I, class... Types>
constexpr tuple_element_t<I, tuple<Types...>>&&
    get(tuple<Types...>&&) noexcept;

```



```

get(tuple<Types...>&&) noexcept;
template <size_t I, class... Types>
constexpr const tuple_element_t<I, tuple<Types...>>&
get(const tuple<Types...>&) noexcept;
template <size_t I, class... Types>
constexpr const tuple_element_t<I, tuple<Types...>>&&
get(const tuple<Types...>&&) noexcept;
template <class T, class... Types>
constexpr T& get(tuple<Types...>& t) noexcept;
template <class T, class... Types>
constexpr T&& get(tuple<Types...>&& t) noexcept;
template <class T, class... Types>
constexpr const T& get(const tuple<Types...>& t) noexcept;
template <class T, class... Types>
constexpr const T&& get(const tuple<Types...>&& t) noexcept;
[...]
```

3. Change 23.3.1 [sequences.general]/2 "Header <array> synopsis" as depicted:

```

[...]
```

```

template <size_t I, class T, size_t N>
constexpr T& get(array<T, N>&) noexcept;
template <size_t I, class T, size_t N>
constexpr T&& get(array<T, N>&&) noexcept;
template <size_t I, class T, size_t N>
constexpr const T& get(const array<T, N>&) noexcept;
template <size_t I, class T, size_t N>
constexpr const T&& get(const array<T, N>&&) noexcept;
[...]
```

4. Change 20.3.4 [pair.astuple] as depicted:

```

template<size_t I, class T1, class T2>
constexpr tuple_element_t<I, pair<T1, T2>>&
get(pair<T1, T2>& p) noexcept;
template<size_t I, class T1, class T2>
constexpr const tuple_element_t<I, pair<T1, T2>>&
get(const pair<T1, T2>& p) noexcept;
```

~~-3- Returns: If I == 0 returns p.first; if I == 1 returns p.second; otherwise the program is ill-formed.~~

```

template<size_t I, class T1, class T2>
constexpr tuple_element_t<I, pair<T1, T2>>&&
get(pair<T1, T2>&& p) noexcept;
template<size_t I, class T1, class T2>
constexpr const tuple_element_t<I, pair<T1, T2>>&&
get(const pair<T1, T2>&& p) noexcept;
```

~~-4- Returns: If I == 0 returns a reference to std::forward<T1>&&(p.first); if I == 1 returns a reference to std::forward<T2>&&(p.second); otherwise the program is ill-formed.~~

```

template <class T, class U>
constexpr T& get(pair<T, U>& p) noexcept;
template <class T, class U>
constexpr const T& get(const pair<T, U>& p) noexcept;
```

~~-5- Requires: T and U are distinct types. Otherwise, the program is ill-formed.~~

~~-6- Returns: get<0>(p);~~

```

template <class T, class U>
constexpr T&& get(pair<T, U>&& p) noexcept;
template <class T, class U>
constexpr const T&& get(const pair<T, U>&& p) noexcept;
```

-7- Requires: T and U are distinct types. Otherwise, the program is ill-formed.

-8- Returns: A reference to p.first.get<0>(std::move(p));

```

template <class T, class U>
constexpr T& get(pair<U, T>& p) noexcept;
template <class T, class U>
constexpr const T& get(const pair<U, T>& p) noexcept;
```

~~-9- Requires: T and U are distinct types. Otherwise, the program is ill-formed.~~

~~-10- Returns: get<1>(p);~~

```
template <class T, class U>
constexpr T&& get(pair<U, T>&& p) noexcept;
template <class T, class U>
constexpr const T&& get(const pair<U, T>&& p) noexcept;
```

-11- Requires: τ and u are distinct types. Otherwise, the program is ill-formed.

-12- Returns: A reference to $p.second$.`get<I>(<std::move(p));`

5. Change 20.4.2.6 [tuple.elem] as depicted:

```
template <size_t I, class... Types>
constexpr tuple_element_t<I, tuple<Types...> >& get(tuple<Types...>& t) noexcept;
```

~~-1- Requires: $I < \text{sizeof}...(Types)$. The program is ill-formed if I is out of bounds.~~

~~-2- Returns: A reference to the i th element of t , where indexing is zero-based.~~

```
template <size_t I, class... Types>
constexpr tuple_element_t<I, tuple<Types...> >&& get(tuple<Types...>&& t) noexcept; // Note A
```

~~-3- Effects: Equivalent to return `std::forward<typename tuple_element<I, tuple<Types...> >::type&&>(get<I>(t))`;~~

~~-4- Note: if a τ in $Types$ is some reference type $x\&$, the return type is $x\&$, not $x\&\&$. However, if the element type is a non-reference type τ , the return type is $\tau\&$.~~

```
template <size_t I, class... Types>
constexpr tuple_element_t<I, tuple<Types...> > const& get(const tuple<Types...>& t) noexcept; // Note B
template <size_t I, class... Types>
constexpr const tuple_element_t<I, tuple<Types...> >&& get(const tuple<Types...>&& t) noexcept;
```

-5- Requires: $I < \text{sizeof}...(Types)$. The program is ill-formed if I is out of bounds.

-6- Returns: A **const** reference to the i th element of t , where indexing is zero-based.

-?- [Note A: if a τ in $Types$ is some reference type $x\&$, the return type is $x\&$, not $x\&\&$. However, if the element type is a non-reference type τ , the return type is $\tau\&$. — end note]

-7- [Note B: Constness is shallow. If a τ in $Types$ is some reference type $x\&$, the return type is $x\&$, not **const** $x\&$. However, if the element type is non-reference type τ , the return type is **const** $\tau\&$. This is consistent with how constness is defined to work for member variables of reference type. — end note]

```
template <class T, class... Types>
constexpr T& get(tuple<Types...>& t) noexcept;
template <class T, class... Types>
constexpr T&& get(tuple<Types...>&& t) noexcept;
template <class T, class... Types>
constexpr const T& get(const tuple<Types...>& t) noexcept;
template <class T, class... Types>
constexpr const T&& get(const tuple<Types...>&& t) noexcept;
```

-8- Requires: The type τ occurs exactly once in $Types...$. Otherwise, the program is ill-formed.

-9- Returns: A reference to the element of t corresponding to the type τ in $Types...$

[...]

6. Change 23.3.2.9 [array.tuple] as depicted:

```
template <size_t I, class T, size_t N>
constexpr T& get(array<T, N>& a) noexcept;
```

~~-3- Requires: $I < N$. The program is ill-formed if I is out of bounds.~~

~~-4- Returns: A reference to the i th element of a , where indexing is zero-based.~~

```
template <size_t I, class T, size_t N>
constexpr T&& get(array<T, N>&& a) noexcept;
```

~~-5- Effects: Equivalent to return `std::move(get<I>(a))`;~~

```
template <size_t I, class T, size_t N>
constexpr const T& get(const array<T, N>& a) noexcept;
template <size_t I, class T, size_t N>
```

```
constexpr const T&& get(const array<T, N>&& a) noexcept;
```

-6- *Requires*: $I < N$. The program is ill-formed if I is out of bounds.

-7- *Returns*: A `const` reference to the i th element of a , where indexing is zero-based.

2486. `mem_fn()` should be required to use perfect forwarding

Section: 20.9.2 [func.require] **Status:** [Ready](#) **Submitter:** Stephan T. Lavavej **Opened:** 2015-03-27 **Last modified:** 2015-05-08

View other [active issues](#) in [func.require].

View all other [issues](#) in [func.require].

View all issues with [Ready](#) status.

Discussion:

20.9.2 [func.require]/4 defines "simple call wrapper" and "forwarding call wrapper". Only `mem_fn()` is specified to be a "simple call wrapper", by 20.9.11 [func.memfn]/1: "A simple call wrapper (20.9.1) fn such that the expression $fn(t, a2, \dots, aN)$ is equivalent to $INVOKE(pm, t, a2, \dots, aN)$ (20.9.2)."

This suggests, but doesn't outright state, that perfect forwarding is involved. It matters for PMFs like $R(T::*)(Arg)$ where Arg is passed by value — if the `mem_fn()` wrapper's function call operator takes Arg by value, an extra copy/move will be observable. We should require perfect forwarding here.

[2015-05, Lenexa]

Move to Immediate.

Proposed resolution:

This wording is relative to N4296.

1. Change 20.9.2 [func.require] as depicted [*Editorial remark*: This simply adds "A simple call wrapper is a `forwarding` call wrapper", then moves the sentence. — *end of remark*]:

-4- Every call wrapper (20.9.1) shall be `MoveConstructible`. ~~A simple call wrapper is a call wrapper that is `CopyConstructible` and `CopyAssignable` and whose copy constructor, move constructor, and assignment operator do not throw exceptions.~~ A forwarding call wrapper is a call wrapper that can be called with an arbitrary argument list and delivers the arguments to the wrapped callable object as references. This forwarding step shall ensure that rvalue arguments are delivered as rvalue-references and lvalue arguments are delivered as lvalue-references. A simple call wrapper is a forwarding call wrapper that is `CopyConstructible` and `CopyAssignable` and whose copy constructor, move constructor, and assignment operator do not throw exceptions. [*Note*: In a typical implementation [...] — *end note*]

2487. `bind()` should be `const`-overloaded, not `cv`-overloaded

Section: 20.9.10.3 [func.bind.bind] **Status:** [Ready](#) **Submitter:** Stephan T. Lavavej **Opened:** 2015-03-27 **Last modified:** 2015-05-08

View other [active issues](#) in [func.bind.bind].

View all other [issues](#) in [func.bind.bind].

View all issues with [Ready](#) status.

Discussion:

The Standard currently requires `bind()` to return something with a `cv`-overloaded function call operator. `const` is great, but `volatile` is not. First, the Library almost always ignores `volatile`'s existence (with `<type_traits>` and `<atomic>` being rare exceptions). Second, implementations typically store bound arguments in a tuple, but `get()` isn't overloaded for `volatile` tuple. Third, when a bound argument is a `reference_wrapper`, we have to call `tid.get()`, but that won't compile for a `volatile` `reference_wrapper`. Finally, `const` and `volatile` don't always have to be handled symmetrically — for example, lambda function call operators are `const` by default, but they can't ever be `volatile`.

Implementers shouldn't be required to provide `cv`-overloading here. (They can provide it as a conforming extension if they want.)

[2015-05, Lenexa]

JW: why would a reference_wrapper be volatile?

STL: if a bound argument is a reference_wrapper then in a volatile-qualified operator() that member will be volatile so you can't call get() on it

STL: worded like this it's a conforming extension to keep overloading on volatile

HH: libc++ doesn't overload on volatile

JW: libstdc++ does overload for volatile

MC: move to Ready and bring motion on Friday

10 in favor, none opposed

Proposed resolution:

This wording is relative to N4296.

1. Change 20.9.10.3 [func.bind.bind] as depicted:

```
template<class F, class... BoundArgs>
    unspecified bind(F&& f, BoundArgs&&... bound_args);
```

-2- *Requires:* is_constructible<FD, F>::value shall be true. For each Ti in BoundArgs, is_constructible<TiD, Ti>::value shall be true. *INVOKE*(fd, w1, w2, ..., wN) (20.9.2) shall be a valid expression for some values w1, w2, ..., wN, where N == sizeof...(bound_args). **The cv-qualifiers cv of the call wrapper g, as specified below, shall be neither volatile nor const volatile.**

[...]

```
template<class R, class F, class... BoundArgs>
    unspecified bind(F&& f, BoundArgs&&... bound_args);
```

-6- *Requires:* is_constructible<FD, F>::value shall be true. For each Ti in BoundArgs, is_constructible<TiD, Ti>::value shall be true. *INVOKE*(fd, w1, w2, ..., wN) shall be a valid expression for some values w1, w2, ..., wN, where N == sizeof...(bound_args). **The cv-qualifiers cv of the call wrapper g, as specified below, shall be neither volatile nor const volatile.**

[...]

2489. mem_fn() should be noexcept

Section: 20.9.11 [func.memfn] **Status:** [Ready](#) **Submitter:** Stephan T. Lavavej **Opened:** 2015-03-27 **Last modified:** 2015-05-08

View all other [issues](#) in [func.memfn].

View all issues with [Ready](#) status.

Discussion:

mem_fn() is wide contract and doesn't do anything that could throw exceptions, so it should be marked noexcept.

Note that mem_fn() is perfectly happy to wrap a null PMF/PMD, it just can't be invoked later. This is exactly like std::function, which can be constructed from null PMFs/PMDs. Therefore, mem_fn() will remain wide contract forever.

[2015-05, Lenexa]

Move to Immediate.

Proposed resolution:

This wording is relative to N4296.

1. Change 20.9 [function.objects] p2 "Header <functional> synopsis" as depicted:

```
[...]
// 20.9.11, member function adaptors:
template<class R, class T> unspecified mem_fn(R T::* pm) noexcept;
[...]
```

2. Change 20.9.11 [func.memfn] as depicted:

```
template<class R, class T> unspecified mem_fn(R T::* pm) noexcept;
```

[...]

~~4- Throws: Nothing.~~

[2492](#). Clarify requirements for comp

Section: 25.4 [alg.sorting] **Status:** [Ready](#) **Submitter:** Anton Savin **Opened:** 2015-04-14 **Last modified:** 2015-05-08

View all other [issues](#) in [alg.sorting].

View all issues with [Ready](#) status.

Discussion:

N4296 25.4 [alg.sorting]/3 reads:

For all algorithms that take Compare, there is a version that uses operator< instead. That is, comp(*i, *j) != false defaults to *i < *j != false. For algorithms other than those described in 25.4.3 to work correctly, comp has to induce a strict weak ordering on the values.

So it's not specified clearly what happens if comp or operator< don't induce a strict weak ordering. Is it undefined or implementation-defined behavior? It seems that it should be stated more clearly that the behavior is undefined.

[2015-05, Lenexa]

Move to Immediate.

Proposed resolution:

This wording is relative to N4431.

1. Change 25.4 [alg.sorting]/3 to the following:

For all algorithms that take Compare, there is a version that uses operator< instead. That is, comp(*i, *j) != false defaults to *i < *j != false. For algorithms other than those described in 25.4.3 to work correctly, comp shall have to induce a strict weak ordering on the values.

[2494](#). [fund.ts.v2] ostream_joiner needs noexcept

Section: X [iterator ostream.joiner] **Status:** [Ready](#) **Submitter:** Nate Wilson **Opened:** 2015-05-03 **Last modified:** 2015-05-08

View all issues with [Ready](#) status.

Discussion:

Addresses: fund.ts.v2

In Library Fundamentals 2 [N4481](#), [iterator ostream.joiner], all operations are no-ops other than the assignment operator.

So, they should be marked as noexcept.

[2015-05, Lenexa]

Move to Immediate.

Proposed resolution:

This wording is relative to N4481 in regard to fundamental-ts-2 changes.

1. Change class template ostream_joiner synopsis, [iterator ostream.joiner] p2, as indicated:

```
namespace std {
namespace experimental {
inline namespace fundamentals_v2 {

template <class DelimT, class charT = char, class traits = char_traits<charT> >
class ostream_joiner {
public:
    [...]
    ostream_joiner<DelimT, charT, traits>& operator*() noexcept;
```

```

        ostream_joiner<DelimT, charT, traits>& operator++() noexcept;
        ostream_joiner<DelimT, charT, traits>& operator++(int) noexcept;
        [...]
    };

} // inline namespace fundamentals_v2
} // namespace experimental
} // namespace std

```

2. Change [iterator ostream.joiner.ops] p3+5, as indicated:

```

ostream_joiner<DelimT, charT, traits>& operator*() noexcept;

[...]

ostream_joiner<DelimT, charT, traits>& operator++() noexcept;
ostream_joiner<DelimT, charT, traits>& operator++(int) noexcept;

```

2495. There is no such thing as an *Exception Safety* element

Section: 20.8.2.2.1 [util.smartptr.shared.const] **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2015-05-05 **Last modified:** 2015-08-03

View all other [issues](#) in [util.smartptr.shared.const].

View all issues with [Tentatively Ready](#) status.

Discussion:

20.8.2.2.1 [util.smartptr.shared.const] includes several "Exception safety" elements, but that is not one of the elements defined in 17.5.1.4 [structure.specifications]. We should either define what it means, or just move those sentences into the *Effects* clause.

[2015-06, Telecom]

Move to Tentatively Ready.

Proposed resolution:

This wording is relative to N4431.

1. Change 20.8.2.2.1 [util.smartptr.shared.const] as follows:

```

template<class Y> explicit shared_ptr(Y* p);

```

[...]

-4- *Effects*: Constructs a shared_ptr object that owns the pointer p. If an exception is thrown, delete p is called.

[...]

~~-7- *Exception safety*: If an exception is thrown, delete p is called.~~

```

template <class Y, class D> shared_ptr(Y* p, D d);
template <class Y, class D, class A> shared_ptr(Y* p, D d, A a);
template <class D> shared_ptr(nullptr_t p, D d);
template <class D, class A> shared_ptr(nullptr_t p, D d, A a);

```

[...]

-9- *Effects*: Constructs a shared_ptr object that owns the object p and the deleter d. The second and fourth constructors shall use a copy of a to allocate memory for internal use. If an exception is thrown, d(p) is called.

[...]

~~-12- *Exception safety*: If an exception is thrown, d(p) is called.~~

[...]

```

template <class Y> explicit shared_ptr(const weak_ptr<Y>& r);

```

[...]

-24- *Effects*: Constructs a `shared_ptr` object that shares ownership with `r` and stores a copy of the pointer stored in `r`. **If an exception is thrown, the constructor has no effect.**

[...]

~~-27- *Exception safety*: If an exception is thrown, the constructor has no effect.~~

```
template <class Y, class D> shared_ptr(unique_ptr<Y, D>&& r);
```

[...]

-29- *Effects*: Equivalent to `shared_ptr(r.release(), r.get_deleter())` when `D` is not a reference type, otherwise `shared_ptr(r.release(), ref(r.get_deleter()))`. **If an exception is thrown, the constructor has no effect.**

~~-30- *Exception safety*: If an exception is thrown, the constructor has no effect.~~

2500. [fund.ts.v2] fundts.memory.smartptr.shared.obs/6 should apply to cv-unqualified void

Section: X [memory.smartptr.shared.obs] **Status:** [Tentatively Ready](#) **Submitter:** Jeffrey Yasskin **Opened:** 2015-05-11 **Last modified:** 2015-09-14

View all issues with [Tentatively Ready](#) status.

Discussion:

Addresses: fund.ts.v2

S. B. Tam reported this [here](#).

[N3920](#) changed `operator*()` in [util.smartptr.shared.obs] as:

Remarks: When τ is **an array type or cv-qualified** void, it is unspecified whether this member function is declared. ...

This excludes cv-unqualified void, which is probably unintended.

[2015-09-11, Telecom]

Move to Tentatively Ready

Proposed resolution:

1. In the [library fundamentals v2](#), [memory.smartptr.shared.obs] p2, change as indicated:

Remarks: When τ is an array type or **(possibly cv-qualified)** void, it is unspecified whether this member function is declared. If it is declared, it is unspecified what its return type is, except that the declaration (although not necessarily the definition) of the function shall be well formed.

2510. Tag types should not be DefaultConstructible

Section: 18.6 [support.dynamic], 20.2 [utility], 20.3.5 [pair.piecewise], 20.7.2 [memory.syn], 20.7.6 [allocator.tag], 30.4 [thread.mutex] **Status:** [Tentatively Ready](#) **Submitter:** Ville Voutilainen **Opened:** 2015-06-13 **Last modified:** 2015-08-03

View all other [issues](#) in [support.dynamic].

View all issues with [Tentatively Ready](#) status.

Discussion:

`std::experimental::optional`, for certain reasons, specifies its `nullopt` type to not be `DefaultConstructible`. It doesn't do so for its tag type `in_place_t` and neither does the standard proper for any of its tag types. That turns out to be very unfortunate, consider the following:

```
#include <memory>
#include <array>

void f(std::array<int, 1>, int) {} // #1
```



```
void f(std::allocator_arg_t, int) {} // #2
```

```
int main()
{
    f({}, 666); // #3
}
```

The call at #3 is ambiguous. What's even worse is that if the overload #1 is removed, the call works just fine. The whole point of a tag type is that it either needs to be mentioned in a call or it needs to be a forwarded argument, so being able to construct a tag type like that makes no sense.

Making the types have an *explicit default constructor* might have helped, but [CWG 1518](#) is going against that idea.

[optional.nullopt]/3 solves this problem for nullopt:

Type nullopt_t shall not have a default constructor. It shall be a literal type. Constant nullopt shall be initialized with an argument of literal type.

[2015-06, Telecom]

Move to Tentatively Ready.

Proposed resolution:

This wording is relative to N4527.

1. In 18.6 [support.dynamic]/1, change the header <new> synopsis:

```
[...]
struct nothrow_t {}; see below
extern const nothrow_t nothrow;
[...]
```

2. Add a new paragraph after 18.6 [support.dynamic]/1 (*following* the header <new> synopsis):

?- Type nothrow_t shall not have a default constructor.

3. In 20.2 [utility]/2, change the header <utility> synopsis:

```
[...]
// 20.3.5, pair piecewise construction
struct piecewise_construct_t {}; see below
constexpr piecewise_construct_t piecewise_construct{ unspecified };
[...]
```

4. Add a new paragraph after 20.2 [utility]/2 (*following* the header <utility> synopsis):

?- Type piecewise_construct_t shall not have a default constructor. It shall be a literal type. Constant piecewise_construct shall be initialized with an argument of literal type.

5. In 20.3.5 [pair.piecewise], apply the following edits:

```
struct piecewise_construct_t {};
constexpr piecewise_construct_t piecewise_construct{ unspecified };
```

6. In 20.7.2 [memory.syn]/1, change the header <memory> synopsis:

```
[...]
// 20.7.6, allocator argument tag
struct allocator_arg_t {}; see below
constexpr allocator_arg_t allocator_arg{ unspecified };
[...]
```

7. Add a new paragraph after 20.7.2 [memory.syn]/1 (*following* the header <memory> synopsis):

?- Type allocator_arg_t shall not have a default constructor. It shall be a literal type. Constant allocator_arg shall be initialized with an argument of literal type.

8. In 20.7.6 [allocator.tag], apply the following edits:

```
namespace std {
    struct allocator_arg_t {};
    constexpr allocator_arg_t allocator_arg{ unspecified };
}
```

Editorial drive-by: `piecewise_construct_t` is written, in 20.3.5 [pair.piecewise] like

```
struct piecewise_construct_t { };  
constexpr piecewise_construct_t piecewise_construct{};
```

whereas other tag types such as `allocator_construct_t` are, in e.g. 20.7.6 [allocator.tag], written like

```
namespace std {  
    struct allocator_arg_t { };  
    constexpr allocator_arg_t allocator_arg{};  
}
```

We should decide whether or not to write out the `std` namespace in such paragraphs. I would suggest not to write it out.

9. In 30.4 [thread.mutex]/1, change the header `<mutex>` synopsis:

```
[...]  
struct defer_lock_t { }; see below  
struct try_to_lock_t { }; see below  
struct adopt_lock_t { }; see below  
  
constexpr defer_lock_t defer_lock { unspecified };  
constexpr try_to_lock_t try_to_lock { unspecified };  
constexpr adopt_lock_t adopt_lock { unspecified };  
[...]
```

10. Add three new paragraphs after [thread.mutex]/1 (following the header `<mutex>` synopsis):

?- Type `defer_lock_t` shall not have a default constructor. It shall be a literal type. Constant `defer_lock` shall be initialized with an argument of literal type.

?- Type `try_to_lock_t` shall not have a default constructor. It shall be a literal type. Constant `try_to_lock` shall be initialized with an argument of literal type.

?- Type `adopt_lock_t` shall not have a default constructor. It shall be a literal type. Constant `adopt_lock` shall be initialized with an argument of literal type.

2515. [fund.ts.v2] Certain comparison operators of `observer_ptr` do not match synopsis

Section: X [memory.observer.ptr.special] **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2015-07-07 **Last modified:** 2015-08-03

View all issues with [Tentatively Ready](#) status.

Discussion:

Addresses: fund.ts.v2

In [N4529](#) [memory.observer.ptr.special] paragraphs 15, 17 and 19, the `>`, `<=` and `>=` operators of `observer_ptr` are shown as

```
template <class W>  
bool operator>(observer_ptr<W> p1, observer_ptr<W> p2);
```

whereas in [header.memory.synop] they are shown as

```
template <class W1, class W2>  
bool operator>(observer_ptr<W1> p1, observer_ptr<W2> p2);
```

Given that the specification of `operator<` took special care to support heterogeneous types, presumably the second version is intended.

[2015-07, Telecom]

Move to Tentatively Ready.

Proposed resolution:

This wording is relative to [N4529](#).

1. Edit X [memory.observer.ptr.special] as indicated:

```
-15- template <class W1, class W2>
bool operator>(observer_ptr<W1> p1, observer_ptr<W2> p2);

[...]

-17- template <class W1, class W2>
bool operator<=(observer_ptr<W1> p1, observer_ptr<W2> p2);

[...]

-19- template <class W1, class W2>
bool operator>=(observer_ptr<W1> p1, observer_ptr<W2> p2);
```

2517. [fund.ts.v2] Two propagate_const assignment operators have incorrect return type

Section: X [propagate_const.assignment] **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2015-07-08 **Last modified:** 2015-08-03

View all issues with [Tentatively Ready](#) status.

Discussion:

Addresses: fund.ts.v2

[N4529](#) [propagate_const.assignment] depicts the two operator=s described as returning by value. This is obviously incorrect. The class synopsis correctly shows them as returning by reference.

[2015-06, Telecom]

Move to Tentatively Ready.

Proposed resolution:

This wording is relative to [N4529](#).

1. Edit [propagate_const.assignment] as indicated:

```
-1- template <class U>
constexpr propagate_const& operator=(propagate_const<U>&& pu)

[...]

-5- template <class U>
constexpr propagate_const& operator=(U&& u)
```

2526. [fund.ts] Incorrect precondition for experimental::function::swap

Section: 20.9.12.2.2 [func.wrap.func.mod] **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2015-08-04 **Last modified:** 2015-09-14

View all other issues in [func.wrap.func.mod].

View all issues with [Tentatively Ready](#) status.

Discussion:

Addresses: fund.ts

20.9.12.2.2 [func.wrap.func.mod] says that the precondition of swap is

```
this->get_memory_resource() == other->get_memory_resource()
```

That compares two pointers and so requires the memory resource used by *this and other to be the exact same object. That doesn't seem correct (for one, it would essentially outlaw swapping all functions constructed with "plain" allocators, since they would each have their own resource_adaptor object and so the pointers will not compare equal). Presumably the intent is to compare the memory_resources for equality.

Also, other is a reference, not a pointer.

[2015-09-11, Telecom]

Move to Tentatively Ready

Proposed resolution:

This wording is relative to [N4480](#).

1. Edit 20.9.12.2.2 [func.wrap.func.mod] as indicated:

```
void swap(function& other);
```

-2- *Requires*: `*this->get_memory_resource() == *other->get_memory_resource()`.

-3- *Effects*: Interchanges the targets of `*this` and `other`.

-4- *Remarks*: The allocators of `*this` and `other` are not interchanged.